

标题与作者

Hongzhi Zang^{1,*}, Mingjie Wei^{6,2,*}, Si Xu³, Yongji Wu⁵, Zhen Guo³, Yuanqing Wang^{4,3}, Hao Lin³, Peih

¹Tsinghua University ²Zhongguancun Academy ³Inf

* 共

2026 年 8 月 17 日

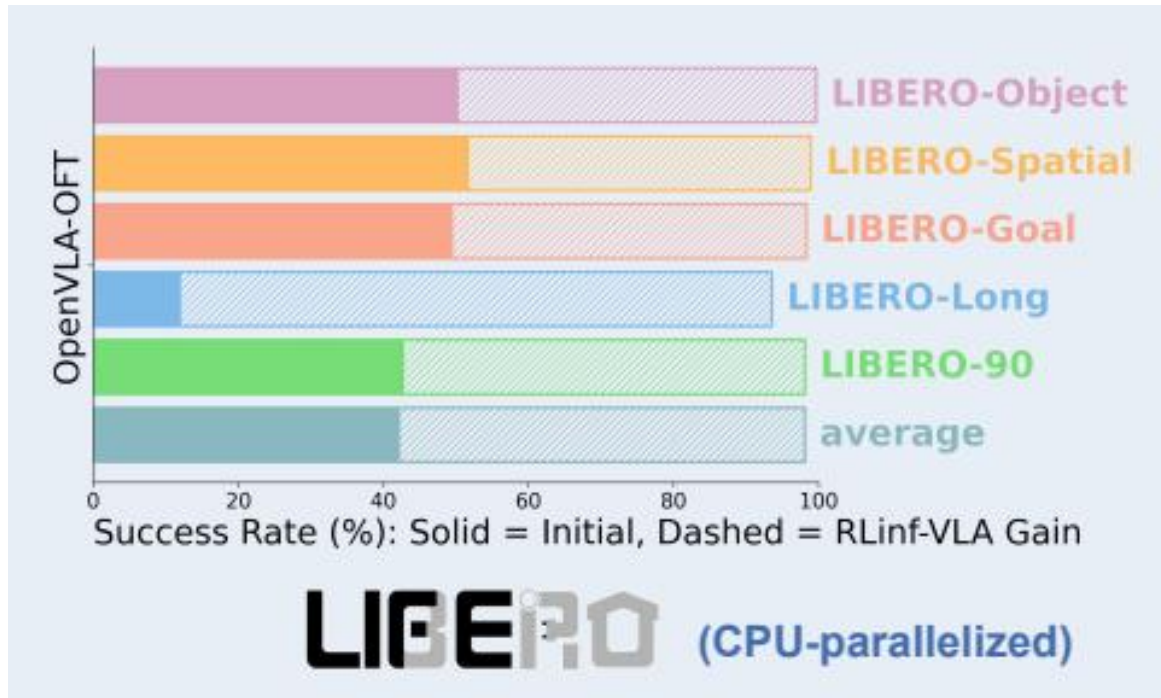


图 1: 标题与作者页

1 RLinf-VLA: 用于 Vision-Language-Action 模型强化学习的统一高效框架

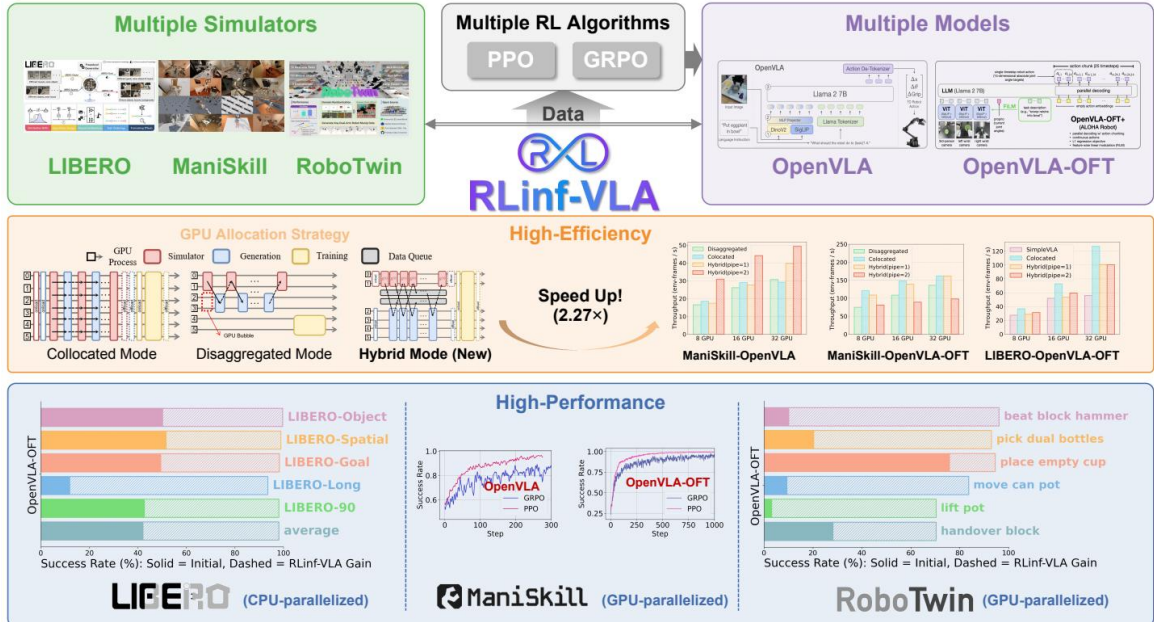


图 2: RLinf-VLA 相关示意图。

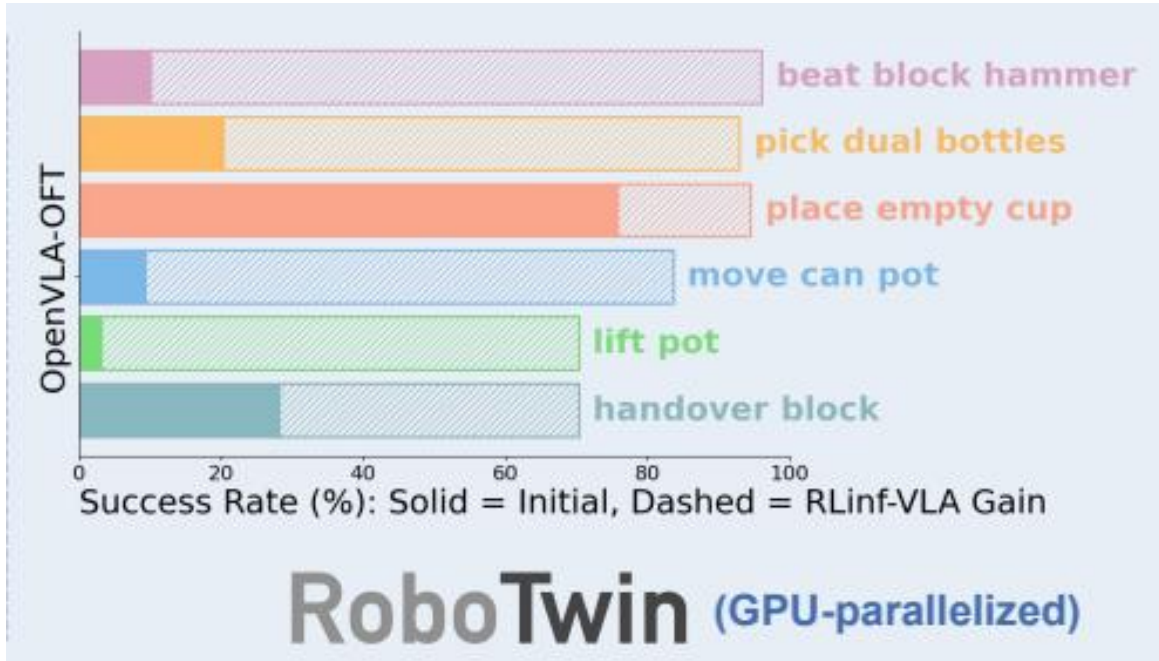


图 3: RLinf-VLA 总览。RLinf-VLA 基于统一接口构建, 可无缝支持多种 VLA 架构、多种 RL 算法以及不同模拟器。框架提供三种 GPU 资源分配模式: collocated、disaggregated, 以及一种新的 hybrid 模式; 与 baseline 相比, 训练速度提升了 2.27 $\times$ 。其中, 一个单模型在 130 个 LIBERO 任务上取得了 98.11% 的成功率 (success rate), 另一个单模型在 25 个 ManiSkill 任务上取得了 97.66% 的成功率。对于 RoboTwin, 6 个任务专用模型在 6 个任务上的平均成功率达到 84.63%。

近期研究表明, 强化学习 (RL) 能够通过交互进一步提升 Vision-Language-Action (VLA) 模型的任务表现, 展现出较大潜力。然而, 现有工作仍较为割裂: 一方面, 缺乏可在不同架构与算法之间进行公平比较的统一平台; 另一方面, 也缺少支持大规模训练的高效系统设计。为此, 我们提出 RLinf-VLA, 一个面向 VLA 模型可扩展 RL 训练的统一且高效的框架。

RLinf-VLA 通过统一接口, 对多样化的 VLA 架构、RL 算法以及异构模拟器进行标准化集成, 从而提升了系统的可扩展性与可复现性。为进一步提高效率, 该框架为 RL pipeline 中的 rendering、inference 和 training 设计了灵活的资源分配架构。特别地, RLinf-VLA 提出了一种 hybrid 的细粒度 pipeline 资源分配策略, 在 ManiSkill 上实现了 $1.61\times$ – $1.88\times$ 的训练加速。

基于该框架训练得到的模型在多个具身智能基准上表现出色: 在 130 个 LIBERO 任务上达到 98.11% 的成功率, 在 25 个 ManiSkill 任务上达到 97.66% 的成功率, 并在 6 个 RoboTwin 任务上取得 84.63% 的平均成功率。此外, RLinf-VLA 还总结出一套适用于基于 RL 的 VLA 训练的有效实践。我们期望 RLinf-VLA 能够成为具身智能研究中的基础性框架, 推动高效、统一且可复现的研究发展。

2 引言

Vision-Language-Action (VLA) 模型代表了一类新一代基础模型, 旨在统一感知、语言理解与具身控制 [28, 10]。这类方法通常先利用基于互联网规模数据预训练的 vision-language model, 再在大规模、异构的机器人 demonstration 数据集上进一步训练 [33, 19], 从而能够将原始传感器观测与自然语言指令直接映射为机器人动作。该范式已在广泛任务上展现出较强的泛化能力 [44, 20, 26, 46, 37, 2, 18]。

然而, VLA 模型在实际部署时通常仍需要经过 post-training, 以缓解训练数据分布与部署环境之间的分布失配问题。若缺少有效的 post-training, 当模型遇到新的状态、指令或执行条件时, 其性能往往会显著下降。

强化学习 (reinforcement learning, RL) 近年来已成为愈发重要的 post-training 范式, 因为它能够通过试错交互直接优化任务的累积奖励 [41]。与另一类常用的 post-training 方法——监督微调 (supervised fine-tuning, SFT) ——相比, RL 不仅能够突破有限专家 demonstration 的覆盖范围进行探索, 还能使 policy 具备纠错与自适应行为。近期研究表明, RL fine-tuning 在分布外泛化方面可能优于 SFT, 尤其体现在语义对齐与执行鲁棒性上 [25]。

尽管前景可观, 但当前将 RL 应用于 VLA 模型的研究大多仍停留在小规模或彼此割裂的状态 [25, 27], 这使得面向 VLA 的新 RL 算法开发既昂贵又繁琐。虽然 SimpleVLA-RL [23] 基于原本为大语言模型设计的 RL 代码库 VeRL [39], 实现了面向 VLA 的大规模 RL 训练, 但它缺乏针对具身场景的系统级优化。在这类场景中, 模拟器、模型 inference 与学习过程会竞争 GPU 资源, 因此该局限会约束系统可扩展性, 并显著拉长训练时间。

此外, 在线 RL 需要反复执行并紧密耦合模型与环境之间的交互。如果系统设计没有针对这种执行模式进行专门适配, 就容易产生大量 GPU 空闲时间与 pipeline bubble, 进而导致资源利用效率低下。最后, 现有工作往往采用彼此不同的模型架构、RL 算法和评测协议, 这不仅使公平比较变得困难, 也阻碍了从基于 RL 的 VLA 训练中提炼一般性规律。

为应对上述挑战, 本文提出 RLinf-VLA: 一个面向 Vision-Language-Action 模型可扩展强化学习的统一且高效的框架。我们的主要贡献如下:

- **统一的系统抽象。** RLinf-VLA 提供统一接口, 支持多种机器人模拟器 (如 ManiSkill [43]、LIBERO [24]、RoboTwin [9])、多样化的 VLA 架构 (如 OpenVLA [20]、OpenVLA-OFT [21]、 π [2]), 以及

多种强化学习算法（如 PPO [36]、GRPO [38]）。系统提供三种执行模式，其中包括一种新的混合式 GPU 分配模式，从而能够在异构配置下实现可扩展且可配置的训练。

- **高效的系统与算法设计。** RLinf-VLA 支持多种灵活的 GPU 分配模式，并进一步引入了一种混合细粒度流水线机制。通过改进资源分配，RLinf-VLA 显著提升了训练吞吐，最高可实现 $2.27\times$ 的加速。此外，该框架还集成了一系列算法层面的增强设计，进一步提高了训练效率与稳定性。
- **强大的经验性能与泛化能力。** 使用 RLinf-VLA 完成 RL 训练后，单个模型在 130 个 LIBERO 任务上取得了 98.11% 的成功率（success rate）；另一个单模型在 25 个 ManiSkill 任务上取得了 97.66% 的成功率。在 RoboTwin 上，6 个任务专用模型在 6 个任务上的平均成功率达到 84.63%，对应平均性能提升为 63.75%。这些结果表明，RLinf-VLA 在 post-training 阶段具有很强的性能优势。
- **开放且可扩展的平台。** RLinf-VLA 已作为开源且持续维护的平台发布，为加速、规范化并规模化具身智能中的强化学习研究提供了实用基础。

3 II. RELATED WORK

3.1 A. 面向 VLA 的强化学习

Vision-Language-Action 模型（VLA）[21, 20, 2, 4, 18, 48, 5, 1, 51, 6] 是一类面向机器人场景的多模态基础模型架构。为克服 VLA 中静态模仿学习的局限，强化学习（RL）正被越来越多地引入这一方向。GRAPE [49] 在部分轨迹上采用 Direct Preference Optimization（DPO [34]），以使模型更好地与人类意图对齐。迭代式方法 [14, 42] 将监督微调（SFT）与 RL 阶段结合起来，在训练稳定性与最终性能之间取得平衡。VLA-RL [27] 使用 Proximal Policy Optimization（PPO [36]），相较于其 SFT 版本 [25]，在未见过的物体和环境上展现出显著更强的泛化能力。SimpleVLA-RL [23] 及其相关变体 [42] 则采用 Group Relative Policy Optimization（GRPO [38]），在无需复杂奖励设计的情况下提升训练效率与性能。 π [7] 首次将 policy gradient 算法应用于 flow matching 模型。 $\pi_{0.6}^*$ [17] 提出 RECAP，用于训练二值化 value function 和 VLA。

要进一步推动基于 RL 的 VLA 发展，亟需统一且高效的基础设施；然而，现有实现仍较为割裂 [25, 27]。尽管已经出现了诸如 SimpleVLA-RL [23] 这样的框架，但其整体上仍沿用通用 LLM RL 后训练范式，未能充分考虑具身智能任务在系统级优化和接口可扩展性方面的特定需求。

3.2 B. 仿真器

仿真器（simulator）[22, 29, 30, 32, 31, 12, 3] 提供了可扩展、安全且可控的环境，从而避开物理硬件带来的约束。它们支持大规模并行数据生成与自动 reset，这对于训练和评估具有良好泛化能力的 VLA policy 至关重要。具有代表性的平台包括：面向富物理操纵任务的 ManiSkill [43]，面向指令条件推理的 LIBERO [24]，以及支持双臂任务并采用 domain randomization 的 RoboTwin [9]。因此，仿真器的构建与使用对于 RL 不可或缺。

然而，不同仿真器之间接口不一致，往往需要对代码进行大幅重构。为此，亟需统一的接口，以支持在不同仿真 benchmark 之间无缝切换。

4 III. PRELIMINARY

4.1 A. 强化学习建模

为说明我们训练基础设施的设计动机, 通常将强化学习 (RL) 表述为一个部分可观测马尔可夫决策过程 (POMDP)。POMDP 由七元组 $(\mathcal{S}, \mathcal{A}, P, r, \gamma, \Omega, O)$ 定义, 其中, $\mathcal{S}$ 和 $\mathcal{A}$ 分别表示状态空间与动作空间, $P(s' | s, a)$ 表示状态转移动力学, $r(s, a)$ 表示奖励函数, $\gamma \in [0, 1]$ 为折扣因子。由于智能体无法直接访问底层状态, 它只能通过观测函数 $O(o | s)$ 接收观测 $o \in \Omega$ 。在基于视觉的 manipulation 任务中, 观测通常由多视角图像、语言指令以及机器人的 proprioception 状态构成, 而动作则对应连续的机器人控制指令或 action chunk。

在每个时间步 t , 智能体根据当前观测 $o_t \sim O(\cdot | s_t)$ 采样动作 $a_t \sim \pi_\theta(\cdot | o_t)$ 。随后, 环境执行 a_t , 并通过 P 转移到下一状态 s_{t+1} , 同时生成下一时刻的观测 o_{t+1} 。强化学习的目标是优化参数化 policy π_θ , 使其在轨迹 $\tau = (s_0, a_0, \dots, s_T)$ 上获得的期望累计折扣奖励最大, 即

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right]. \quad (1)$$

4.2 B. 动作表述

与面向 LLM 的 RL 相比, 面向 VLA 的 RL 在动作建模上的一个关键差异在于其层级化的动作表述方式。LLM 通常以单个 token 为操作单位 [38]; 而 VLA policy 往往预测单个动作 [20], 或者预测由多个动作组成的 action chunk, 以保证控制过程的平滑性 [50]。因此, 这里形成了一个三层层级结构: Chunk、Atomic Action 和 Token。每个 chunk 可以包含多个 atomic action, 而每个 atomic action 又由多个 token 构成, 其中每个 token 通常对应机器人动作空间中的一个特定维度, 例如 end-effector 位姿或关节角。

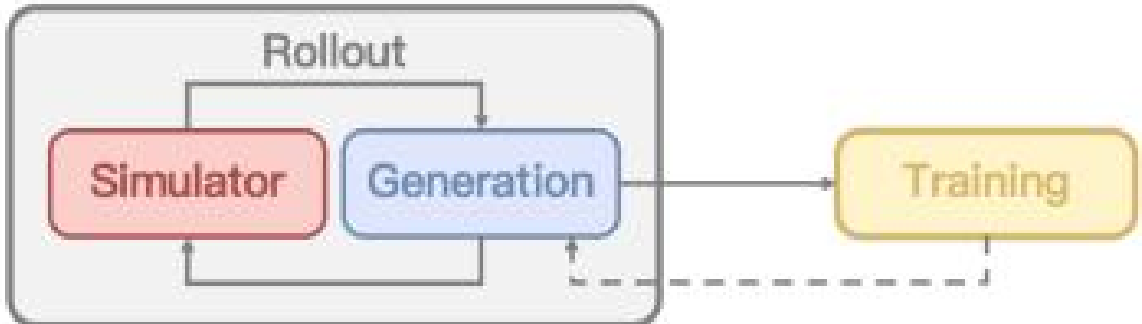


图 4: VLA 模型强化学习 (RL) 流程示意图

4.3 C. VLA 模型的强化学习流程

VLA 模型的强化学习 (RL) 流程由三个彼此独立的组成部分构成: Generation、Simulator 和 Training, 如图 3 所示。GPU 资源主要分配在 rollout 阶段 (即 Generation 与 Simulator) 和优化阶段 (即 Training) 之间。

其交互过程通常如下 [21]: Generation 模块根据当前观测推断一个 action chunk; 随后, Simulator 执行该 action chunk, 并返回最后一个时间步的观测。上述循环会持续进行, 直到完成整条

轨迹的采集；之后，Training 模块利用收集到的数据对 policy 进行更新。

5 IV. METHOD

第 IV-A 节介绍灵活的 GPU 分配策略，用于在不同类型的 simulator 之间实现更高效的资源调度。第 IV-B 节进一步提出统一接口 (Unified Interface)，以无缝集成多种 simulator、VLA 模型与 RL 算法。在此系统设计基础上，第 IV-C 节总结了面向 VLA 模型可扩展 RL 训练的关键设计选择。

5.1 A. GPU 分配策略

用于 VLA 的 RL 训练流水线（见第 III-C 节）在 Training、Generation 和 Simulator 三个组件上的资源需求并不相同。资源瓶颈高度依赖于 simulator 的类型：CPU 并行的 simulator 通常受限于 CPU，GPU 主要用于渲染与 inference；而 GPU 并行的 simulator 则将仿真、渲染和 inference 全部放在 GPU 上执行。后者虽然吞吐更高，但也会在 GPU 显存与算力上带来严重竞争。为了在这些不同设置下尽可能高效地利用资源，必须采用灵活且优化的 GPU 分配策略。我们的框架支持三种灵活且易于配置的分配模式：collocated、disaggregated，以及一种新的 hybrid 模式。

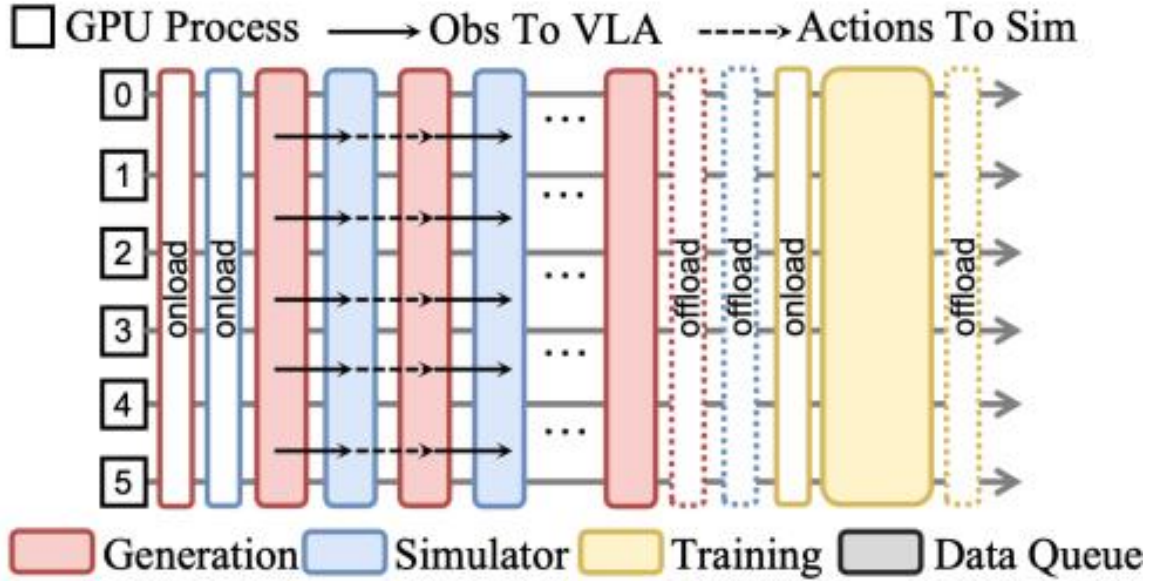


图 5: Collocated 模式。

1) Collocated 模式 在这种模式下（图 4），所有组件都共置在同一组 GPU 上。collocated 模式的原始设计目标，是在任意时刻只让一个组件占用全部 GPU，从而最大化 GPU 利用率。当某个组件完成计算后，就将其从 GPU 卸载到 CPU 内存；等到下一次需要执行该组件的任务时，再将其加载回 GPU。

然而，在 embodiment 场景下，Simulator 和 Generation 都需要使用 GPU 资源，并且二者会以迭代方式频繁交互。这样一来，每次交互都进行 offload-onload 操作会引入巨大的额外开销，实际中难以接受。

因此，我们实现了一种改进版的 collocated 策略：对于 Simulator 和 Generation，只在 rollout 阶段的开始和结束时执行 offload 和 onload，如图所示。该方法确实避免了 rollout 期间频繁的 offload-onload 开销，但仍无法充分利用 GPU 资源，因此依然不是最优方案。由于 Generation 和 Simulator

必须迭代交互，二者会花费大量时间彼此等待；在等待期间，它们虽然占据着 GPU 显存，却并不进行计算，最终造成资源浪费，并限制了系统的可扩展性。

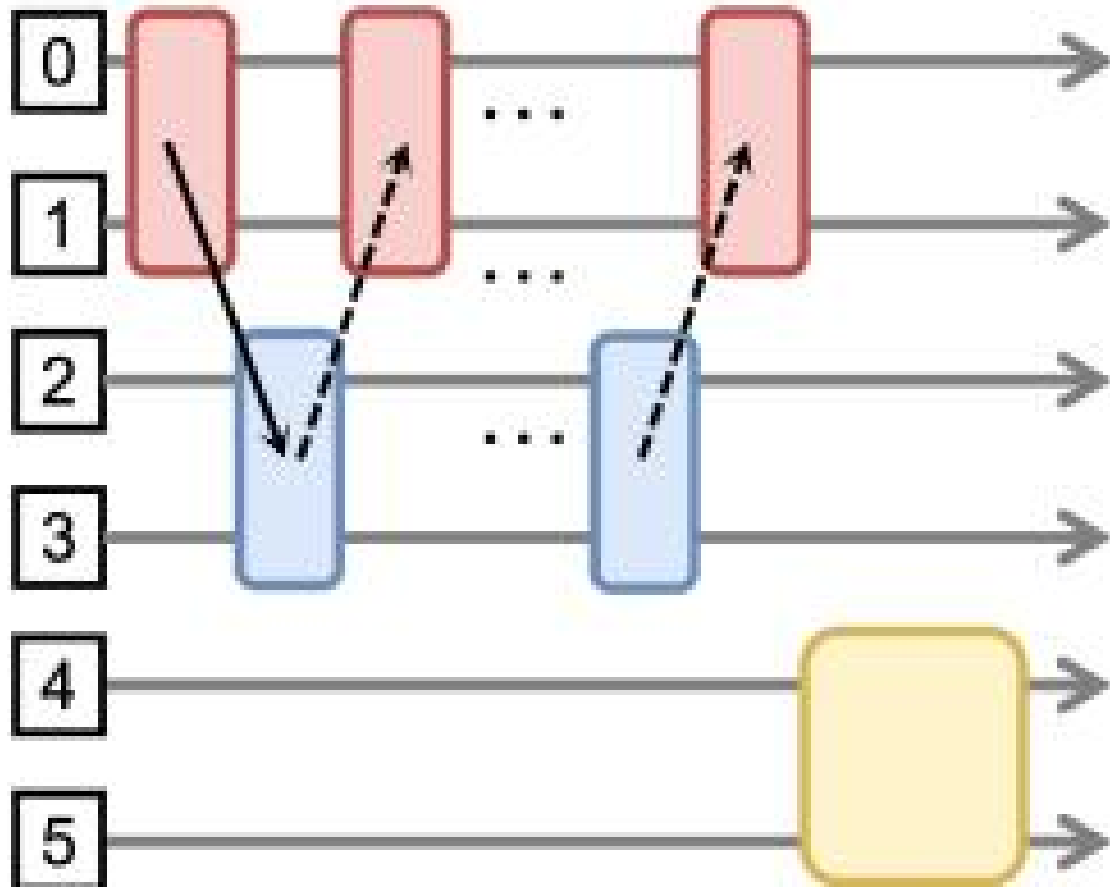


图 6: Disaggregated 模式（图例见图 4）。

2) Disaggregated 模式 在这种模式下，每个组件都被分配到彼此独立的分区中（每个分区也可以包含多张 GPU），不同组件之间没有重叠。图 5 展示了其核心思想。这样做可以保证每个组件都充分使用自己被分配到的资源。

这种模式实现起来较为简单，但由于组件之间存在依赖关系，仍会导致 GPU 利用不足。例如，在图 5 中，GPU 4 和 GPU 5 在 rollout 阶段到 training 开始之前始终处于完全空闲状态。

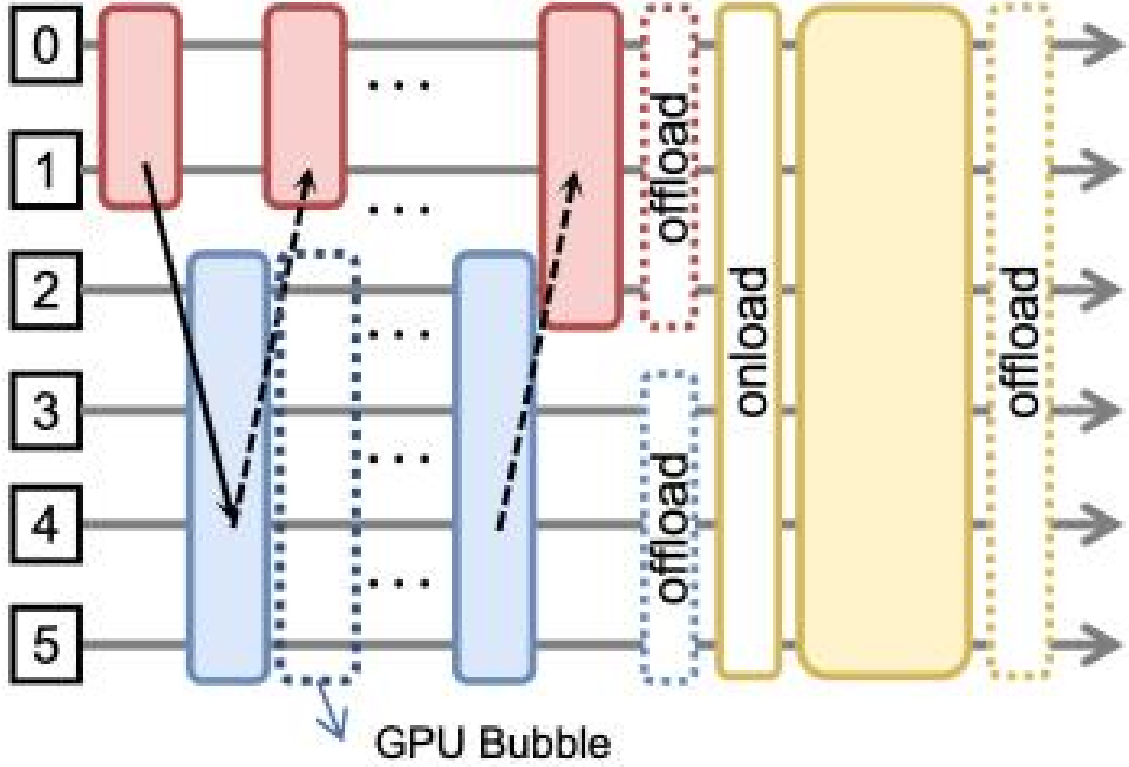


图 7: Hybrid 模式（图例见图 4）。

3) 带细粒度流水化的 Hybrid 模式 为克服上述两种模式的缺点，我们提出了一种 hybrid 模式，其中每个组件都可以灵活选择所使用的 GPU。相比之下，现有的分布式 RL 系统在这方面灵活性较弱。例如，veRL [40] 主要采用 collocated 设计；AReaL [11] 和 SLiME [52] 则采用 disaggregated 设计，但仍然以连续 GPU 块的方式为各组件分配资源。

在我们的 hybrid 模式中，一种典型配置是：将 Generation 和 Simulator 分配到不同的 GPU 分区，同时允许 Training 使用全部 GPU。图 6 给出了示意。然而，即便如此，资源仍然无法被充分利用。例如，Simulator 必须等待 Generation 生成 action，这会使部分 GPU 空闲，从而形成“GPU bubbles”。

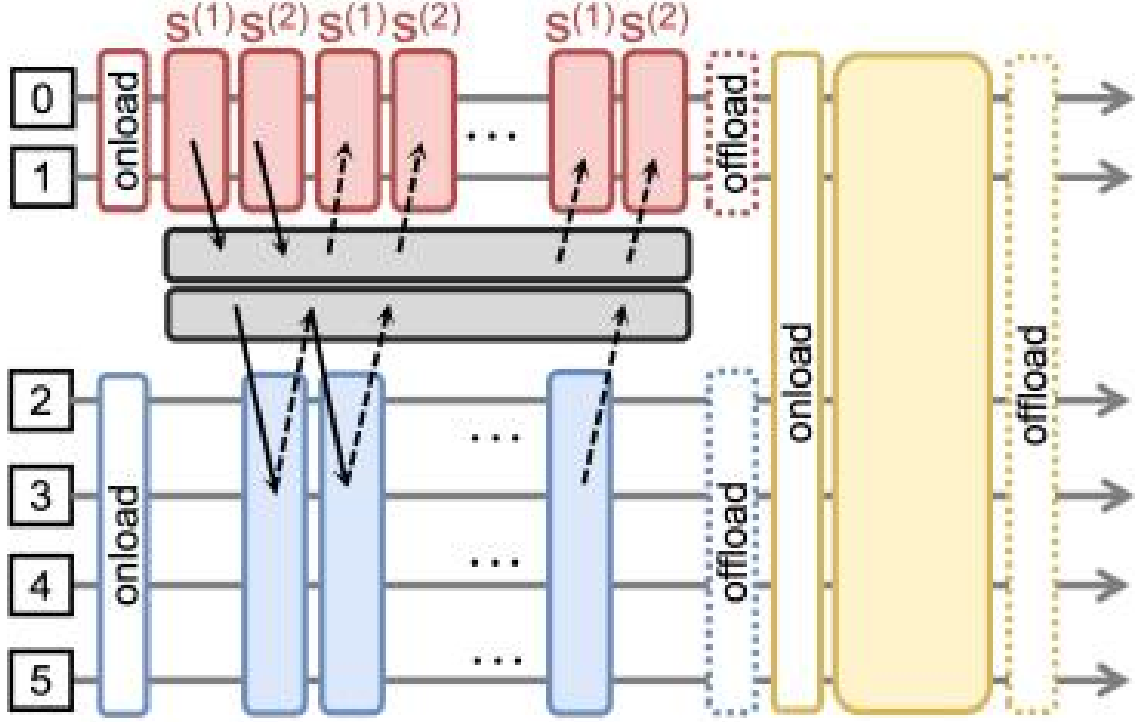


图 8: 带细粒度流水化的 Hybrid 模式 ($k = 2$; 图例见图 4)。

在此基础上, 我们进一步引入细粒度流水化, 以缓解由组件间依赖造成的 bubbles。具体而言, 可以将单张 GPU 上的一个 simulator 实例划分为多个子 simulator, 记为 $S^{(1)}, S^{(2)}, \dots, S^{(k)}$ 。当 $k = 2$ 时, 流水线过程如下, 其中上标表示 simulator 的编号, 下标表示时间步:

1. 在步骤 $t = 0$ 时, $S^{(1)}$ 生成初始观测 $o_0^{(1)}$, 并将其发送给 Generation 组件, 以生成动作 $a_0^{(1)}$ 。
2. 与此同时, $S^{(2)}$ 并行生成 $o_0^{(2)}$ 。

5.2 A. GPU Allocation Strategies (第 2 部分)

3) 当 $a_0^{(1)}$ 准备就绪后, 会将其回传给 $S^{(1)}$, 从而生成下一条观测 $o_1^{(1)}$; 与此同时, actor 还会并行处理 $o_0^{(2)}$ 以生成 $a_0^{(2)}$ 。

这种调度方式使 Simulator 与 Generation 能够并发执行, 在保证正确性的同时减少空闲时间。由此, 我们的框架既避免了 colocated allocation 中频繁 offloading 带来的开销, 也消除了 disaggregated allocation 中出现的 GPU bubble。图 6 给出了采用细粒度流水线 (fine-grained pipelining) 时混合分配模式的示意图, 其中 ($k = 2$)。

4) 自动放置算法 (Auto-Placement Algorithm): 由于 RLinf-VLA 支持多种 GPU 分配模式, 如何自动确定高效的分配策略十分重要。为此, 我们提出了一种轻量级的自动 GPU 放置算法, 将运行时 profiling 与分配优化结合起来。

我们首先给出若干实际假设。给定 N 张 GPU, 我们假设 Training 组件只要处于激活状态, 就会使用全部 N 张 GPU, 因为这通常是计算密集型工作负载下的最优选择。因此, 我们将优化重点放在 Rollout 阶段。该阶段由 Simulator 和 Generation 两个组件构成。我们只考虑在这两个组件之间划分 GPU, 并将流水线深度限制为最多两级。给定 N 张 GPU 和 n_S 个 simulation environments, 我们的目标是确定 GPU 分配 (N_S, N_G) 以及流水线级数 $k \in \{1, 2\}$, 使 rollout 时间最小, 其中

N_S 和 N_G 分别表示分配给 Simulator 和 Generation 组件的 GPU 数量。

Profiling. 我们通过对批大小 $\{2^0, 2^1, \dots, n_S\}$ 进行 profiling, 来估计执行时间函数 $t_G(\cdot)$ 和 $t_S(\cdot)$, 从而得到每个 chunk 的执行延迟估计。

Allocation. 记 m 为每个 epoch 的 chunk 数量。我们求解 $\min_{1 \leq k \leq \{1, 2\}} N_G + N_S = N t_k$, 其中

Collocated [formula]

Pipelined ($k = 2$): t_2 由 $t_G(\frac{n_S}{N_G})$ 和 $t_S(\frac{n_S}{N_S})$ 在流水线执行下进行估计。

5.3 B. 统一接口

为便于复用现有模拟器、模型与算法, RLinf-VLA 提供了一个具有高可扩展性与可伸缩性的统一接口。

1) 支持多种模拟器: RLinf-VLA 支持多种机器人模拟器, 包括 ManiSkill [43]、LIBERO [24] 和 RoboTwin [9]。为此, 我们设计了统一接口, 用于规范不同 backend 下的环境交互, 例如 ManiSkill 与 RoboTwin 的 GPU 并行环境, 以及 LIBERO 的 CPU 并行 wrapper。该接口提供了标准的 Gym 风格 API, 并内置支持 action chunking 与灵活的 episode 终止机制。此外, 我们还定义了统一的 observation 与 action 接口, 以保证与不同算法和模型之间的无缝兼容。

2) 支持多种模型: 得益于跨模拟器统一的数据处理流水线, 我们的框架只要求模型遵循标准化的交互接口, 即可无缝集成现有模型。本文以 OpenVLA [20] 和 OpenVLA-OFT [21] 为例, 展示了对多种 VLA 架构的支持; 二者分别对应单步和多步 action chunking 的配置。为实现高效微调 (fine-tuning), 我们集成了 Low-Rank Adaptation (LoRA) [16]。

3) 支持多种算法: 我们的框架支持多种强化学习算法, 初始重点放在 Proximal Policy Optimization (PPO) [36] 和 Group Relative Policy Optimization (GRPO) [38, 13]。对于 on-policy 强化学习算法, 其核心在于 advantage function 和 loss function。在本框架中, 只需指定这两个函数即可完成算法配置, 无需编写冗余代码。

对于 off-policy 强化学习算法, 其在 VLA 模型上的应用仍是一个开放问题, 尤其是在实现稳定训练方面。尽管如此, 我们的流水线已经具备完整功能, 并且可以扩展到 off-policy 设定。作为初步扩展, RLinf-VLA 支持 DSRL [45], 即 Soft Actor-Critic (SAC [15]) 的一个变体, 可用于微调 π -series 模型。不过, off-policy 方法并非本文的重点, 因此我们不再展开讨论。

5.4 C. 算法的设计选择

面向大模型的 RL 算法, 相比小模型需要做出更复杂的设计选择。对于 VLA, 我们同时借鉴了 LLM 与机器人学中的方法论。在第 IV-C1 节中, 我们先介绍适用于所有算法的一般性特征; 随后在第 IV-C2 节和第 IV-C3 节中, 分别说明 PPO 与 GRPO 的具体设计选择。与我们的代码库风格一致, 这些特性都可以通过修改配置文件中的参数值来方便地配置。

1) 多粒度计算支持 我们的框架支持在多种粒度上进行 advantage estimation 与 log-probability 计算, 从而能够无缝适配不同算法的需求。

对于 advantage estimation, 我们为 action chunk $\mathbf{c}_t = (a_{t,1}, \dots, a_{t,C})$ 提供了两种形式。其一是 **chunk-level** 形式: 将整个 chunk 视为一个宏动作 (macro-action), 并为其分配求和后的 reward 以及共享的 advantage。其二是 **action-level** 形式: 对 chunk 中每个原子动作 $a_{t,i}$ 分别分配独立的 reward 与 advantage。

类似地, 对于 log-probability 计算, 我们支持如下三种对应粒度:

- **Chunk-level:** [formula]
- **Action-level:** [formula]
- **Token-level:** $\pi(d_{t,i,j} | o_t, d_{t,i,:j-1})$

其中, $d_{t,i,j}$ 表示第 i 个原子动作的第 j 个 token。

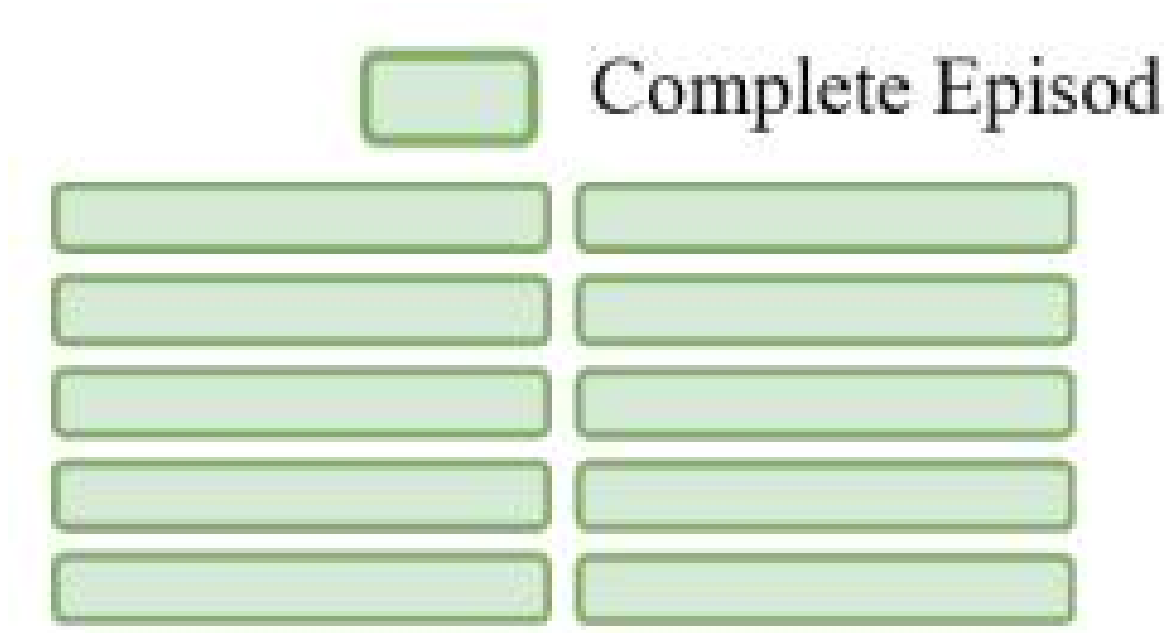


图 9: 三种 rollout 模式示意图。

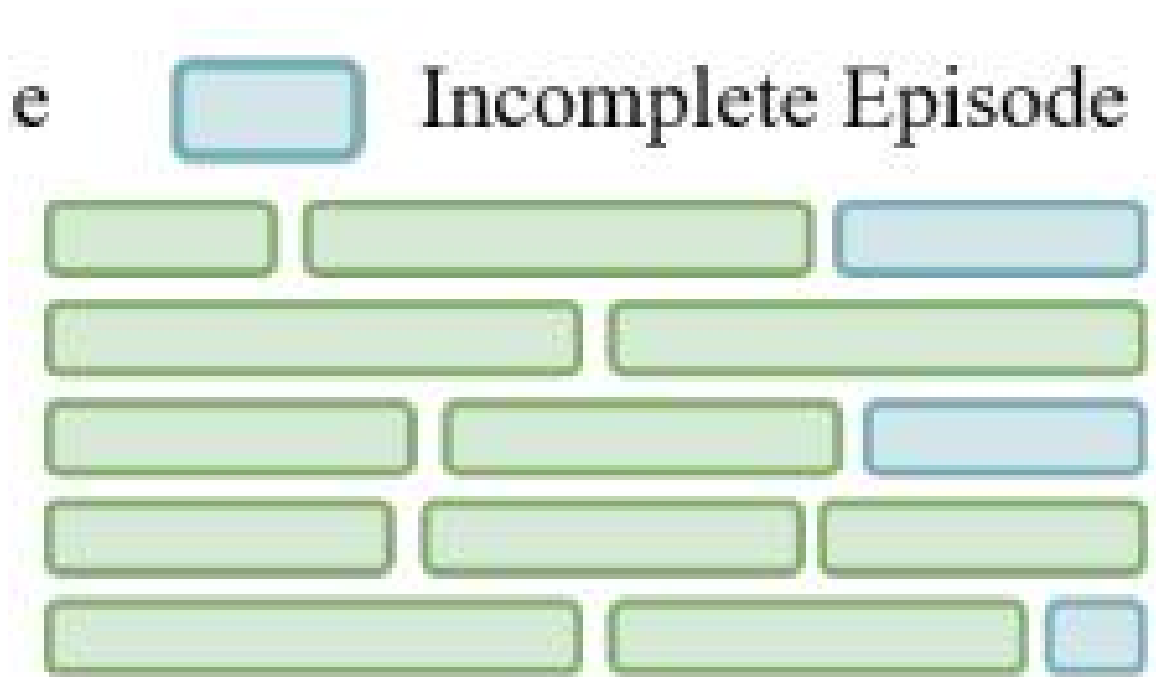


图 10: (a) Fixed Length; (b) Partial Reset。

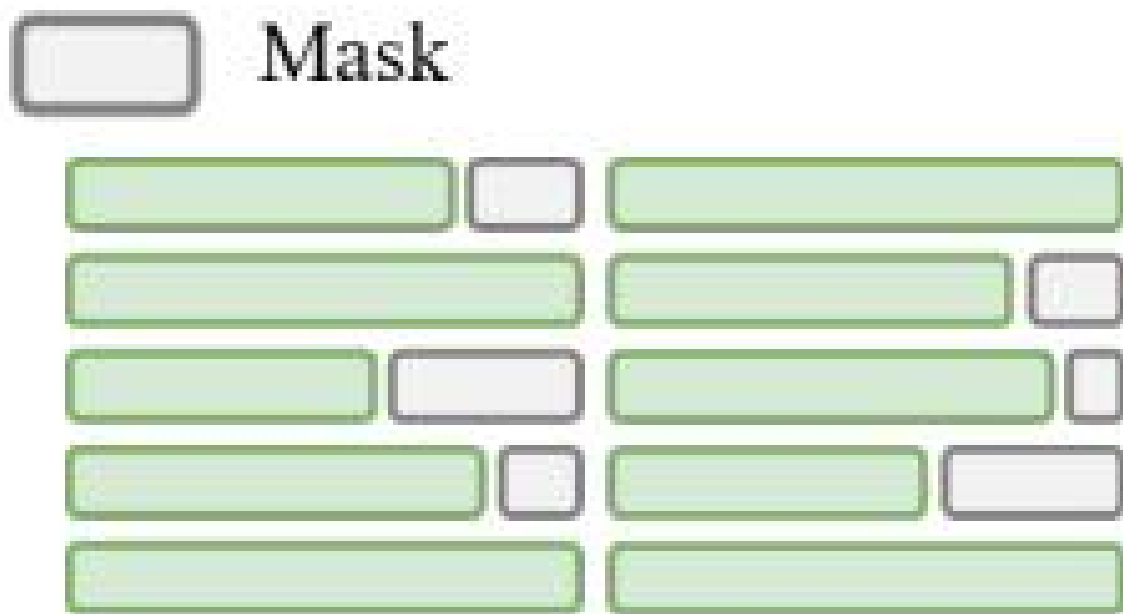


图 11: (c) Valid Action Mask。

2) PPO 的设计选择 将 PPO 扩展到 VLA 模型会带来若干设计挑战。下面总结我们在框架中采用的关键设计决策。

Partial Reset 支持 在 rollout 过程中，一个子环境通常会在达到最大 episode 长度时终止，或者在成功完成任务后终止。对于这类终止情况，我们考虑了两种标准处理策略。

在 **Fixed Episode Length** 模式下，只有当所有子环境都达到最大 episode 长度后，才会统一进行 reset。相对地，在 **Partial Reset** 模式下，每个子环境一旦终止就立即 reset，而不受其他子环境

状态的影响。图 8(a) 和图 8(b) 展示了这两种模式之间的差异。

Critic 设计 将 PPO 适配到 VLA 模型，对 critic 的设计提出了更严格的要求。由于 VLA 模型规模较大，维持一个独立的 critic 网络不仅计算开销高，还会增加 GPU 资源管理的复杂度。因此，我们采用 actor 与 critic 参数共享的策略。遵循 RL4VLA [25]，我们在 VLA 的语言模型组件上附加一个轻量级 value head，用于高效地进行状态价值估计。

Action Chunk 的 Value 如第 IV-C1 节所述，两种 advantage estimation 形式会自然对应两种针对 action chunk $\mathbf{c}_t = (a_{t,1}, a_{t,2}, \dots, a_{t,C})$ 的 value estimation 策略。

在 **chunk-level** 形式下，critic 将整个 chunk 视为一个宏动作，并为其估计单个标量 value，即 $V : \mathbb{R}$ 。相对地，在 **action-level** 形式下，critic 输出一个 C 维 value 向量，为 chunk 中每个原子动作 $\mathbf{a}_{t,i}$ 提供独立的 value 估计，即 $V : \mathbb{R}^C$ 。从经验结果来看，action-level 形式始终能带来更好的性能。

3) GRPO 的设计选择 将 GRPO 迁移到具身任务中会引入一些并不平凡的挑战。为了解决这些问题，我们采用了如下设计选择。

5.5 C. 算法设计选择（第 2 部分）

Valid Action Mask. 尽管 rollout 按固定的 episode 时长执行，但任务往往会提前结束。这就给 policy 优化带来两种可选策略：(1) 无论任务何时完成，都使用完整轨迹；(2) 仅考虑任务完成之前的时间步。

我们的框架同时支持这两种策略，并将后者称为 **Valid Action Mask** 设定（图 8c）。经验结果表明，在 GRPO 设定下，将任务完成之后的时间步 mask 掉，通常能够提升 policy 的性能。

按轨迹长度进行损失归一化。 为了使不同长度的轨迹在优化过程中具有可比的贡献，在 Valid Action Mask 设定下，我们用有效时间步的数量对 policy loss 进行归一化。具体来说，对于一条轨迹 τ_i ，若其包含 T_i^{succ} 个有效时间步，则目标函数中每个时间步的贡献都按 $1/T_i^{\text{succ}}$ 进行缩放。这样可以避免较长轨迹主导梯度，同时促进成功轨迹与部分完成轨迹之间更平衡的学习。

成功率（success rate）过滤器。 受 DAPO [47] 中动态采样策略的启发，我们为 GRPO 引入了成功率过滤器。该过滤器会丢弃这样一些轨迹组：组内所有轨迹要么全部成功，要么全部失败，因为要计算非零 advantage，必须同时存在成功和失败两类结果。实践中，这一机制能够加快收敛，并稳定提升 policy 的性能。

6 V. 实验结果

本节围绕所提出的 RLinf-VLA 框架的有效性，系统考察五个关键问题：

1. **RLinf-VLA 是否具备高性能?**我们在三个具有代表性的测试平台上评估 RLinf-VLA, 包括 LIBERO、ManiSkill 和 RoboTwin。结果表明，相比 base model，RLinf-VLA 可带来约 20–85% 的性能提升，体现出其对大规模多任务学习的强大支撑能力。

2. **RLinf-VLA 是否具备高效率?** 我们在 GPU 并行和 CPU 并行两类 simulator 上对该框架进行了基准测试。实验发现, 最优配置能够显著提升训练吞吐, 其中最高可实现 $1.88\times$ 的加速。

表 1: 三个仿真基准上的评测结果。表中数值为成功率 (success rate, %)。

(a) ManiSkill 上的评测结果							
Method		In-Distribution		OOD Avg.			
OpenVLA (Base)		53.91		39.10			
OpenVLA (RLinf-GRPO)		84.38		75.15			
OpenVLA (RLinf-PPO)		96.09		81.93			
OpenVLA-OFT (Base)		28.13		18.29			
OpenVLA-OFT (RLinf-GRPO)		94.14		60.64			
OpenVLA-OFT (RLinf-PPO)		97.66		77.05			

(b) LIBERO 上的评测结果						
OpenVLA-OFT	Object Spatial		Goal	Long	90	Avg.
Base	50.20	51.61	49.40	11.90	42.67	42.09
RLinf-GRPO	99.67	98.93	98.32	93.55	98.12	98.11

(c) RoboTwin 上的评测结果							
OpenVLA-OFT	Cup	Hammer	Bottles	Can	Pot	Hand	Avg.
Base	75.78	10.15	20.31	9.37	3.13	28.13	24.48
RLinf-GRPO	94.53	96.09	92.96	83.59	70.31	70.31	84.63
SimpleVLA-RL	94.2	87.5	68.3	61.2	64.1	57.8	72.18

这一结果进一步说明, 支持多样化资源分配模式是十分必要的。值得注意的是, 相比现有框架, RLinf-VLA 最高可实现 $2.27\times$ 的加速。

3. **将 PPO 和 GRPO 应用于 VLA 训练时, 有哪些可操作的实践经验?** 通过大规模 ablation study, 我们识别出影响训练性能的关键因素, 并据此给出在 VLA 场景中有效部署 RL 的实践指导。
4. **基于仿真的 RL 能否帮助真实世界部署?** 我们开展了初步的真实世界部署实验, 以检验在仿真中经 RL 改进的 policy 能否迁移到物理机器人上。与仅采用 SFT 的 baseline 相比, 在我们的实验设置下, RL post-training 能够提升真实世界性能, 表明基于仿真的 RL 在具身学习中具有潜在的实际价值。

5. **RLinf-VLA 在模型与算法层面的可扩展性如何？** 我们进一步在更多 VLA 架构和 RL 算法上评估 RLinf-VLA，包括 π -series 模型以及 off-policy 方法。第 X-A 节的结果表明，该框架能够适配多种训练设置，具有良好的可扩展性。

6.1 A. Policy Performance

本节评估 RLinf-VLA policy 在多个 benchmark 上的有效性，重点考察任务成功率 (success rate) 与泛化性能。

实验设置 我们在三个 benchmark 上评估 RLinf-VLA：ManiSkill、LIBERO 和 RoboTwin，完整设置见附录 III.A。

对于 ManiSkill，我们在 25 个 pick-and-place 任务上训练，这些任务包含物体与容器的多种变化。我们遵循 RL4VLA [25] 的 out-of-distribution 评测协议，从视觉、语言和动作三个方面衡量泛化能力。每个子集均在 256 个随机 episode 上评估。

对于 LIBERO，我们采用公开任务组 [24]，包括 LIBERO-Spatial、Object、Goal、Long 和 90，用于训练与评测。不同于仅在单一任务组内训练的做法，我们将 5 个组共计 130 个任务合并，训练一个统一模型，以测试大规模多任务学习能力。性能按任务组汇报；每个任务在 3 个随机种子下、各评测 50 个 episode，并取平均结果。

对于 RoboTwin [9]，我们选取 6 个任务：“beat block hammer”、“move can pot”、“place empty cup”、“pick dual bottles”、“lift pot”和“handover block”。训练时，每个任务使用 1,000 个固定的随机化场景种子；评测时采样 128 个未见过的种子，以检验 out-of-distribution 泛化能力。

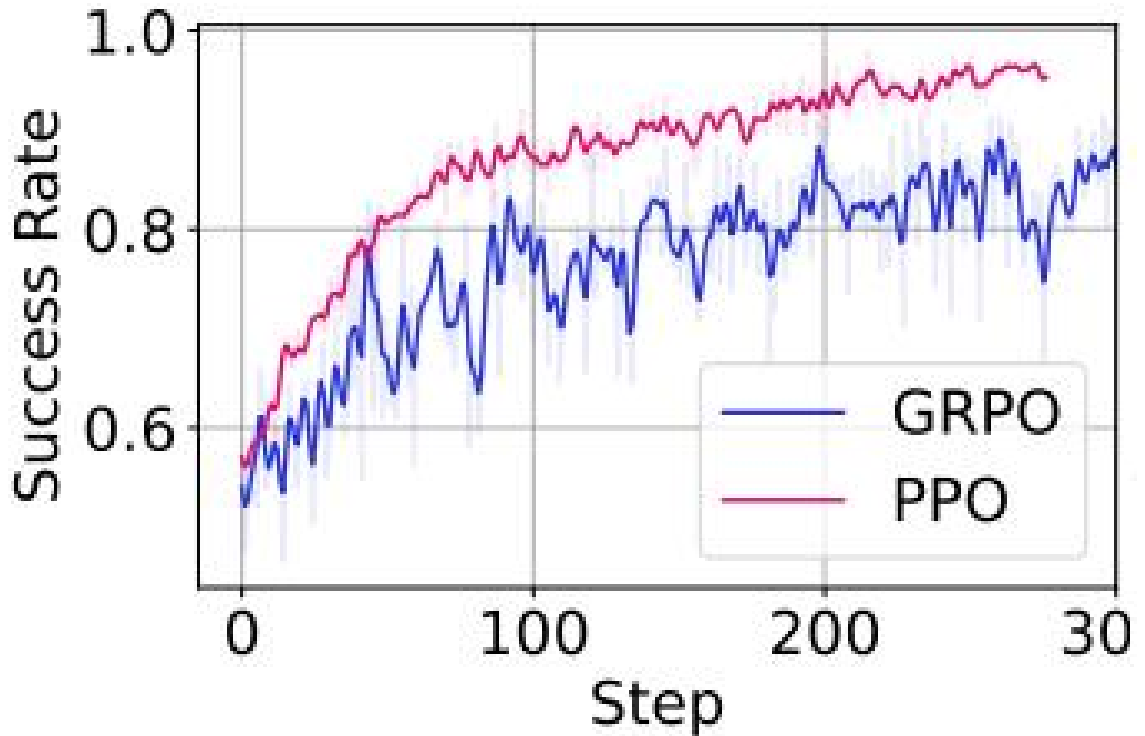


图 12: 不同 benchmark 上的训练曲线。(a) ManiSkill 上的 OpenVLA；(b) ManiSkill 上的 OpenVLA-OFT；(c) LIBERO 上的 OpenVLA-OFT；(d) RoboTwin 上的 OpenVLA-OFT。横轴表示训练 epoch 数，纵轴表示对应的成功率。浅色曲线为原始数据，深色曲线为采用 $\sigma = 1$ 的 Gaussian filter 平滑后的结果。

训练结果 图 9 给出了各实验设置下的训练曲线。图 9(a) 和图 9(b) 展示了 ManiSkill 上的训练表现, 其中使用 OpenVLA 和 OpenVLA-OFT 模型, 并分别通过 PPO 与 GRPO 进行训练。在所有设置下, RL 都带来了显著性能提升, 相比 baseline, 成功率提高了 45%–70%。值得注意的是, 在 ManiSkill 设置中, 无论对于 OpenVLA 还是 OpenVLA-OFT, PPO 都稳定优于 GRPO, 且训练过程更加平稳。

图 9(c) 展示了 OpenVLA-OFT 在 LIBERO 的 130 个任务上采用 GRPO 算法训练得到的曲线。结果表明, 成功率从约 73% 显著提升至 98%, 整体性能增益约为 30%。这一结果表明, RLinf-VLA 中的 GRPO 设计能够有效促进多任务训练。

图 9(d) 给出了 OpenVLA-OFT 在 RoboTwin 上的训练曲线。训练过程整体稳定、波动很小。此外, 即便对于初始成功率极低的任务 (最低仅 3%), 模型最终也能收敛到较高性能。

评测结果 表 I 汇总了 RLinf-VLA 在三个 benchmark 上的结果, 其中 ManiSkill 的详细结果见表 Ia。“OpenVLA (Base)” 和 “OpenVLA-OFT (Base)” 分别表示 OpenVLA 与 OpenVLA-OFT 的 RL base model。

由于 OpenVLA 与 OpenVLA-OFT 的 base 性能不同, 直接比较二者的 OOD 结果并不完全公平。经过 RL 后, OpenVLA 的成功率从 39.10% 提升到 81.93%, 而 OpenVLA-OFT 则从 18.29% 提升到 77.05%。尽管 OpenVLA-OFT 的起点更低, 但其相对增益略大, 这说明 base model 的选择会显著影响最终泛化表现。

表 Ib 报告了 OpenVLA-OFT 在 5 个 LIBERO 任务组共 130 个任务上, 结合 RLinf-VLA 训练后的结果。LIBERO-Object 的成功率达到 93%–99%, 总体平均成功率为 98.11%, 平均提升幅度为 56.02%。需要强调的是, 我们在所有任务上仅使用一个统一模型, 而标准方法通常会针对每个任务组分别训练独立模型。这表明 RLinf-VLA 支持大规模多任务 RL。

RoboTwin 上的 OOD 结果见表 Ic。RLinf-VLA 的平均成功率达到 84.63%, 提升超过 60%。我们还与最相关的 baseline, 即 SimpleVLA-RL [23], 进行了比较。RLinf-VLA 在所有 RoboTwin 任务上都持续优于该方法, 尤其是在 Bottles、Can 和 Hand 等更具挑战性的任务上提升更为明显。这些结果验证了我们系统设计的有效性。

6.2 B. 系统效率

本节通过比较不同的 GPU 分配模式来评估系统效率。实验结果表明, 最优策略会随 simulator 和模型而变化, 这说明灵活的 GPU 分配机制至关重要, 而这正是 RLinf-VLA 的核心特性之一。

实验设置 这里先概述关键设置, 完整细节见附录 III.B。我们从三个 benchmark 中选取了具有代表性的任务进行评测。对于 ManiSkill, 使用 “PutCarrotOnPlateInScene-v2”, 并设置 256 个并行环境。对于 LIBERO, 采用 LIBERO-Long suite; 对于 RoboTwin, 采用 “place empty cup” 任务。对 LIBERO 和 RoboTwin, 并行环境数随节点数线性扩展, 即在 1、2、4 个节点上分别为 64、128 和 256。

我们考虑三种 GPU 分配模式 (见第 IV-A 节): Collocated 在不同组件之间共享 GPU; Disaggregated 将训练与 rollout 分离; Hybrid 则通过将实例拆分为不同阶段, 以流水线方式执行 simulation。比较设置如下: 由于 ManiSkill 缺少多 GPU baseline, 我们将 Collocated 和 Hybrid (pipelined) 模式与一个朴素的 Disaggregated baseline 进行比较。对于 LIBERO 和 RoboTwin, 我们以 SimpleVLA-RL [?] (仅支持 collocated) 作为对比, 并评测我们的 Collocated 和 Hybrid 模式; 由于在 GPU 利用不足时, Hybrid 始终优于 Disaggregated, 因此这里省略后者。

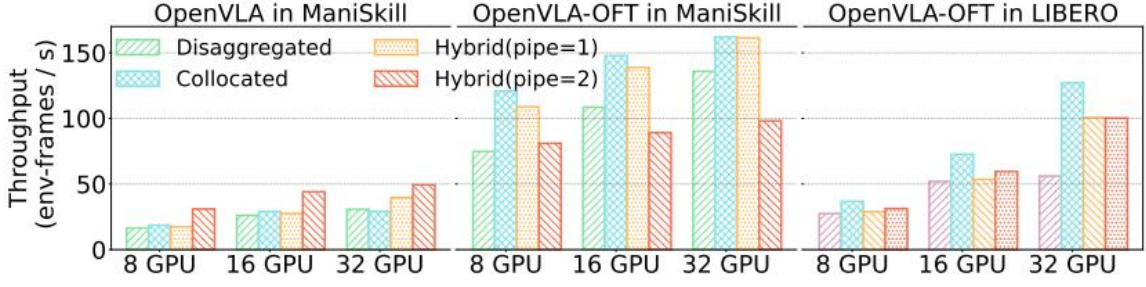


图 13: 不同设置下的吞吐量。

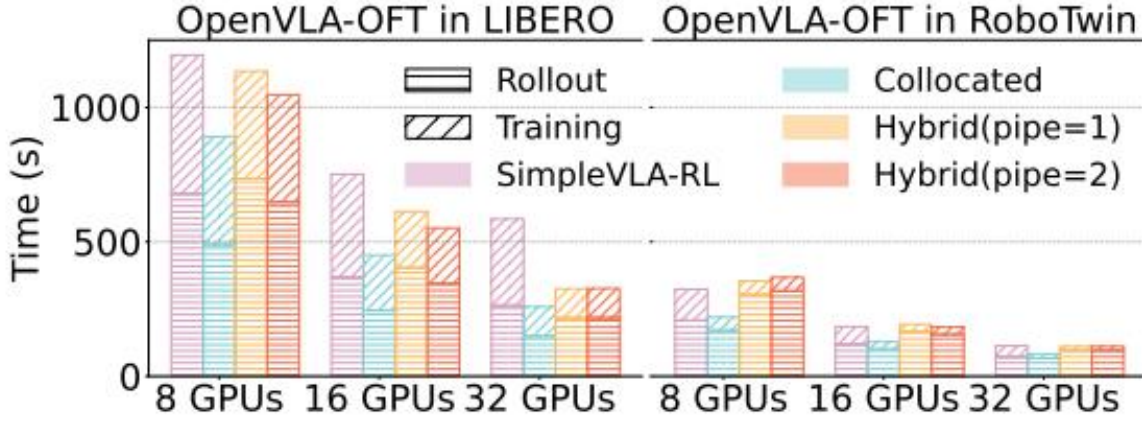


图 14: 延迟分解。图 10: 在 ManiSkill、LIBERO 和 RoboTwin 上评估系统效率, 对比 OpenVLA 与 OpenVLA-OFT。随着流水线阶段数 (“pipe”) 增加, 吞吐量 (总环境帧率) 得到提升。

结果 图 10 给出了在不同集群规模和 GPU 分配模式下, RLinf-VLA 与各 baseline 在 RL 训练中的端到端吞吐量。

从图 10(a) 可以看出, 在 ManiSkill 上使用 OpenVLA 时, RLinf-VLA 在 8 张 GPU、Hybrid (pipe=2) 模式下, 相比 Disaggregated baseline 最高可获得 $1.88\times$ 加速。尽管扩展到更多 GPU 会引入模型加载、offloading 以及状态切换等额外开销, 但细粒度流水线能够减少 GPU 空闲时间, 使其相对 Disaggregated 仍保持 $1.61\times$ – $1.69\times$ 的优势。更深的流水线 (pipe=2 相比 pipe=1) 还能进一步降低 rollout 阶段的 bubble, 从而继续提升性能。

对于 ManiSkill 上的 OpenVLA 以及 ManiSkill 上的 OpenVLA-OFT, 随着 GPU 数量增加, Hybrid (pipe=1) 模式会优于 Collocated 模式。原因在于 ManiSkill 采用 GPU 并行化, rollout 吞吐量会随并行环境数提升, 因此 simulator 的资源分配成为关键。为提高利用率, 我们在 Collocated 模式中启用了 offloading (见第 IV-A1 节), 但通信开销会随着集群规模增大而增长。相比之下, Hybrid 模式将 GPU 在各组件之间均匀切分, 从而减少相互干扰。

而在 LIBERO 和 ManiSkill 上使用 OpenVLA-OFT 时, 现象有所不同: Collocated 和 Hybrid (pipe=1) 优于 Disaggregated 和 Hybrid (pipe=2)。首先, OpenVLA-OFT 生成的是 action chunks, 而 simulator 会按顺序逐步执行它们, 因此生成与执行的比例从 1:1 转变为约 1:15。其次, LIBERO 的 CPU 并行 simulator 成为主要瓶颈。这种不平衡削弱了流水线带来的收益, 使得整体执行过程实际上退化为近似串行。

从图 10(b) 可以看到, 在 LIBERO 和 RoboTwin 上使用 OpenVLA-OFT 时, 即便在 Collocated 设置下, RLinf-VLA 相比 SimpleVLA-RL 仍可取得 $1.34\times$ – $2.27\times$ 的加速。性能提升同时来自 rollout 和

training 两部分。在 rollout 阶段, RLinf-VLA 使用 vectorized environments, 其效率高于 SimpleVLA-RL 的多进程 worker; 在 training 阶段, 自适应通信、去除冗余 log-probability 重计算等系统级优化进一步降低了延迟, 而且这种收益会随着规模增大而更加明显。

小结 不同 simulator 对 GPU 分配模式的需求并不相同。RLinf-VLA 通过支持灵活的多模式执行, 能够在多种不同的运行条件下持续保持较高的系统效率。

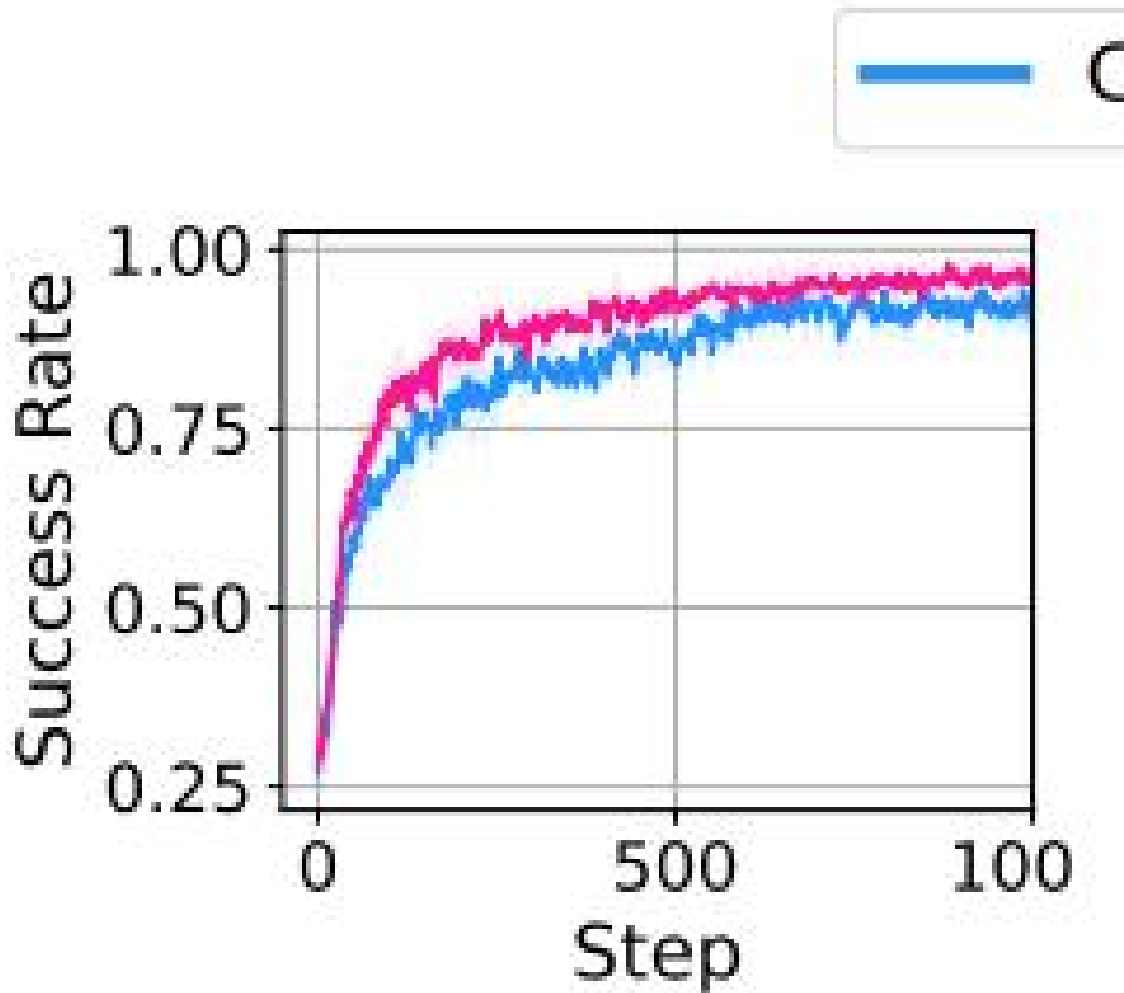


图 15

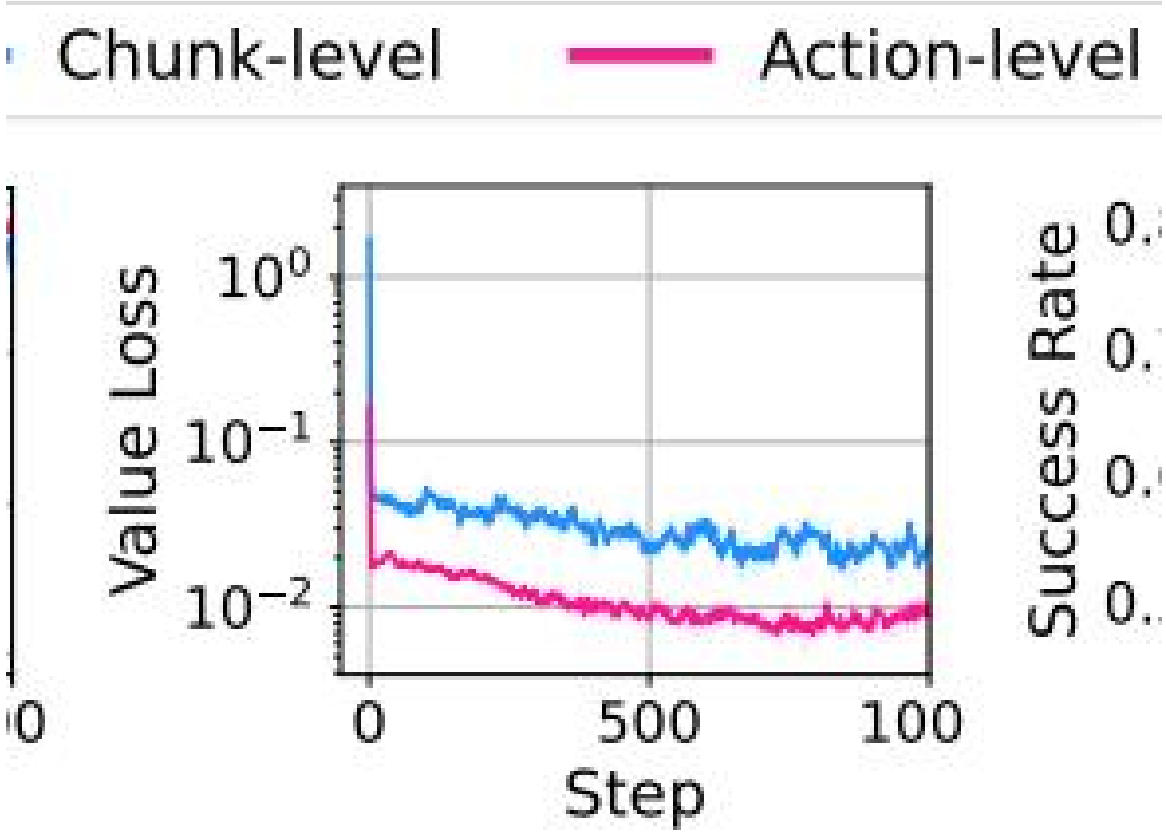


图 16

6.3 B. 系统效率 (第 2 部分)

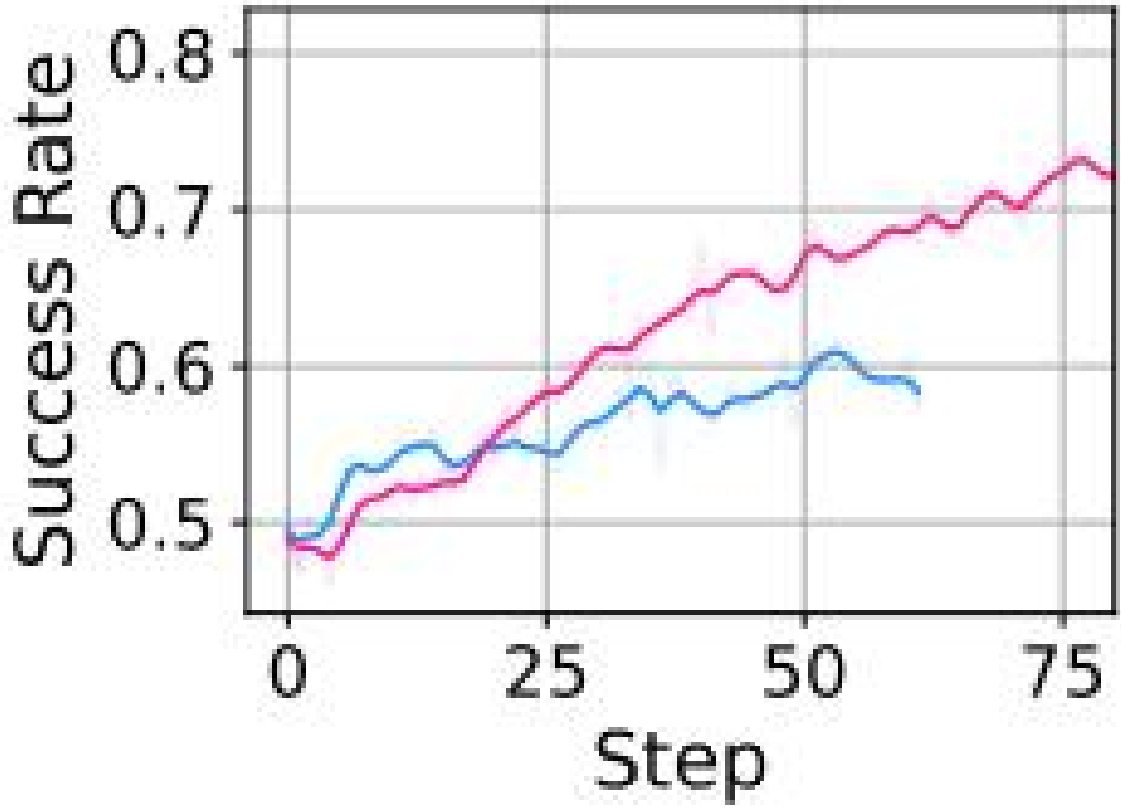


图 17: 在 OpenVLA-OFT 中, 动作级 value estimation 在不同任务上均稳定优于 chunk 级估计。(a) ManiSkill 中的成功率 (success rate); (b) ManiSkill 中的 loss; (c) LIBERO 中的成功率 (success rate)。

6.4 C. 消融实验

本节在不同设置下给出消融实验, 并总结 PPO 与 GRPO 训练中的关键经验。

1) PPO 的经验: 对于带有 action chunk 的 PPO, **action-level** value estimation 优于 **chunk-level** estimation。图 11 展示了在 ManiSkill 的 “PutOnPlateInScene25Main” 任务上, 基于 OpenVLA-OFT 模型、按照第 IV-C2 节所述设置, 对比 action-level 与 chunk-level value estimation 的 PPO 训练曲线。可以看到, action-level estimation 始终带来更高的成功率 (success rate) 和更低的 value loss, 说明其学习更有效, policy 提升也更快。不同 value estimation 粒度之间的性能差异在多种任务上都保持一致, 这一点也得到了 LIBERO-Goal benchmark 上类似结果的印证 (图 11(c))。

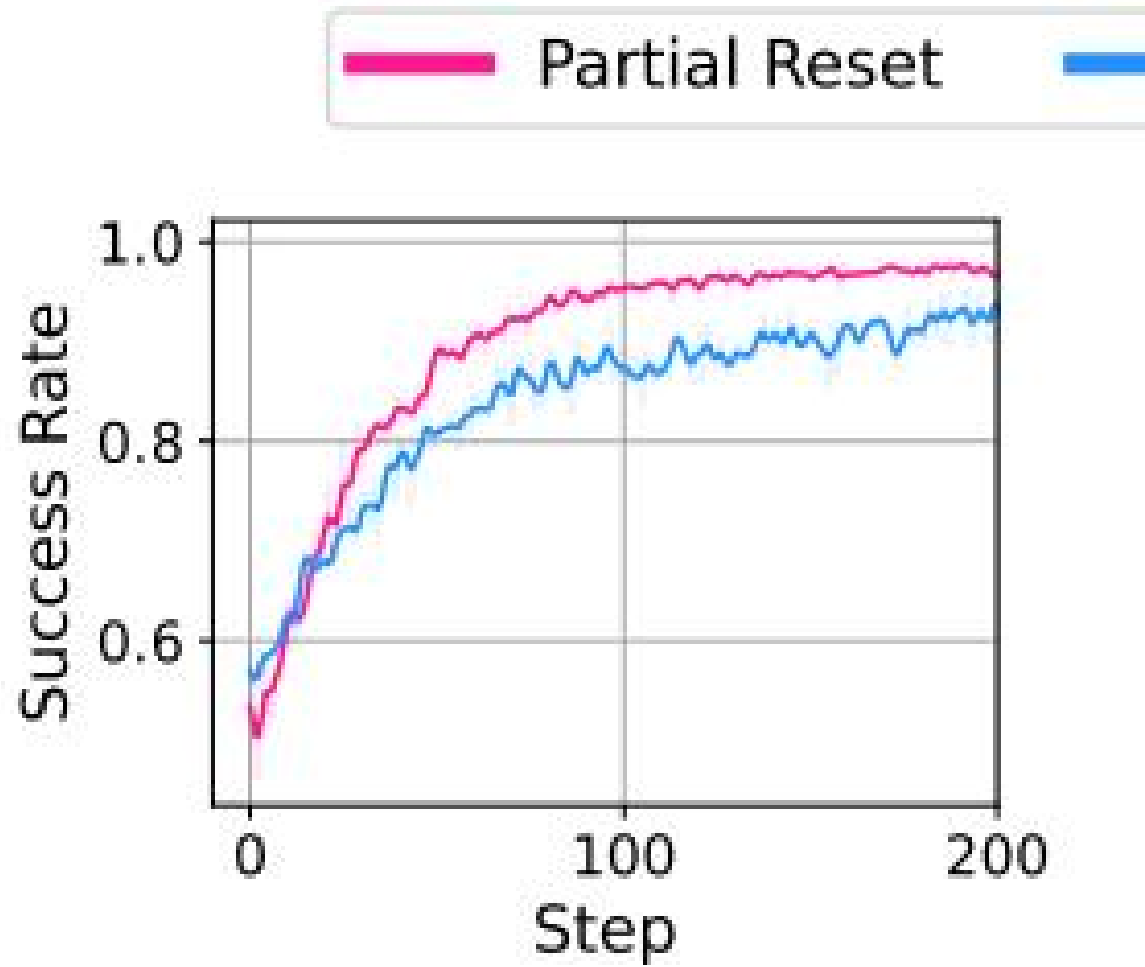


图 18: PPO 中，Partial Reset 在 OpenVLA 上的成功率表现。

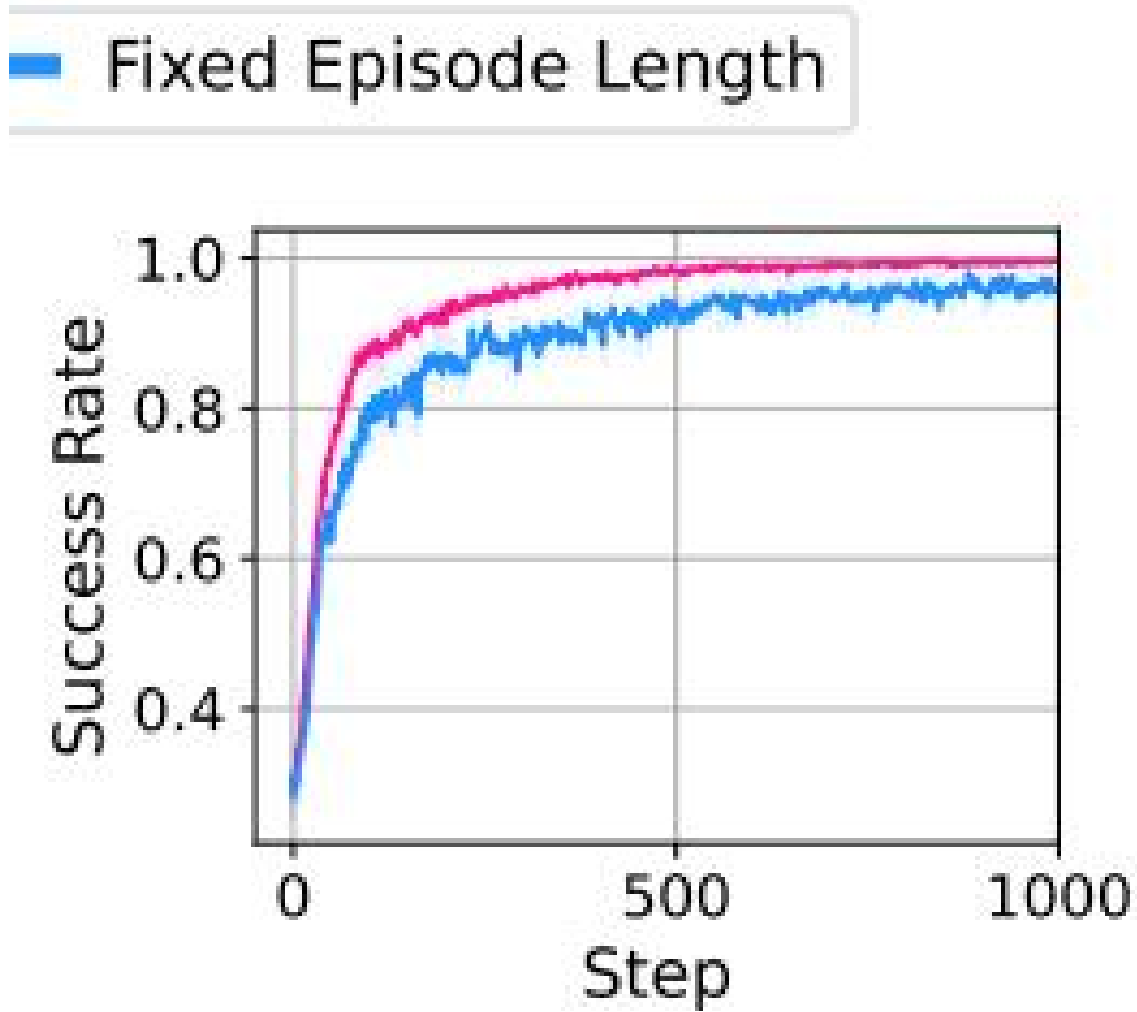


图 19: PPO 中, Partial Reset 在 OpenVLA-OFT 上的成功率表现。(a) OpenVLA 上的成功率; (b) OpenVLA-OFT 上的成功率。Partial reset 始终优于 Fixed Episode Length 模式。

Partial reset 能显著提升样本效率。图 12 给出了第 IV-C2 节中讨论的 Partial Reset 与 Fixed Episode Length 两种 rollout 模式下的 PPO 训练曲线。由于优化目标是在每次 rollout 中至少成功一次 (“success once”), Partial Reset 会带来显著更高的成功率。在相同训练步数下, Partial Reset 的成功率始终高于 Fixed Episode Length 模式。这一现象与模型类型无关, 在 OpenVLA 与 OpenVLA-OFT 上都成立。

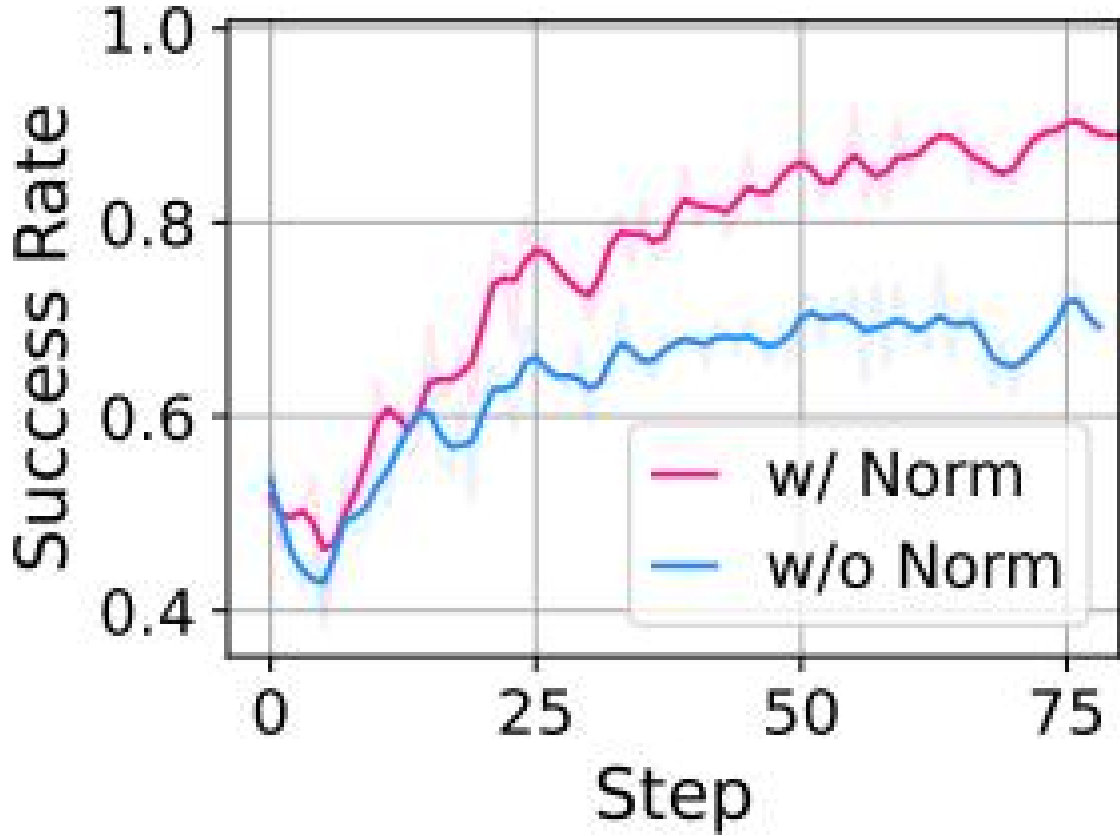


图 20: GRPO 中关于 trajectory length normalization 的消融实验。OpenVLA-OFT 在 LIBERO-Goal 上的成功率。

2) GRPO 的经验: **trajectory length normalization**。如第 IV-C3 节所述, 当 episode 长度不一致时, 按 trajectory 长度对 loss 进行归一化, 目的是减小偏差。图 13 表明, 引入 trajectory length normalization (“w/ Norm”) 相比不做归一化 (“w/o Norm”) 能够显著提升性能。

Valid action mask。我们还研究了第 IV-C3 节提出的 valid action masking 的影响。由于优化目标是 “success once”, 因此在达到成功状态后, 将后续 action 排除在优化之外, 能够提升样本效率, 并避免冗余更新。此外, 使用 valid action mask 后, trajectory 会自然变短, 这也进一步有利于 trajectory length normalization。图 14(a) 展示了在 LIBERO-Goal 上的结果。比较 “w/o Mask, w/o Norm” 与 “w/ Mask, w/o Norm” 两条曲线可以看出, 引入 valid action mask 后, 性能持续更优。而 “w/ Mask, w/ Norm” 曲线进一步说明, 将 mask 与 trajectory length normalization 结合还能带来额外收益。不过, 这些技术的效果依赖于具体任务。在 ManiSkill 设置中, 我们没有观察到 valid action masking 或 trajectory length normalization 带来的明显收益。

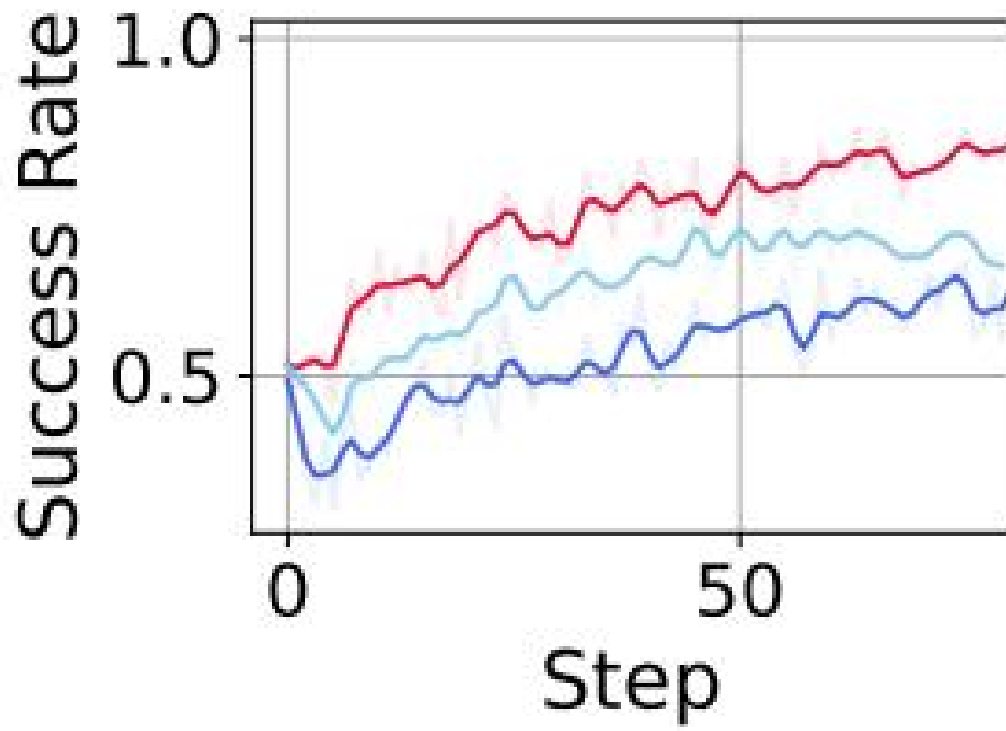


图 21: GRPO 中 valid action mask 的消融实验。(a) LIBERO-Goal 上的成功率。

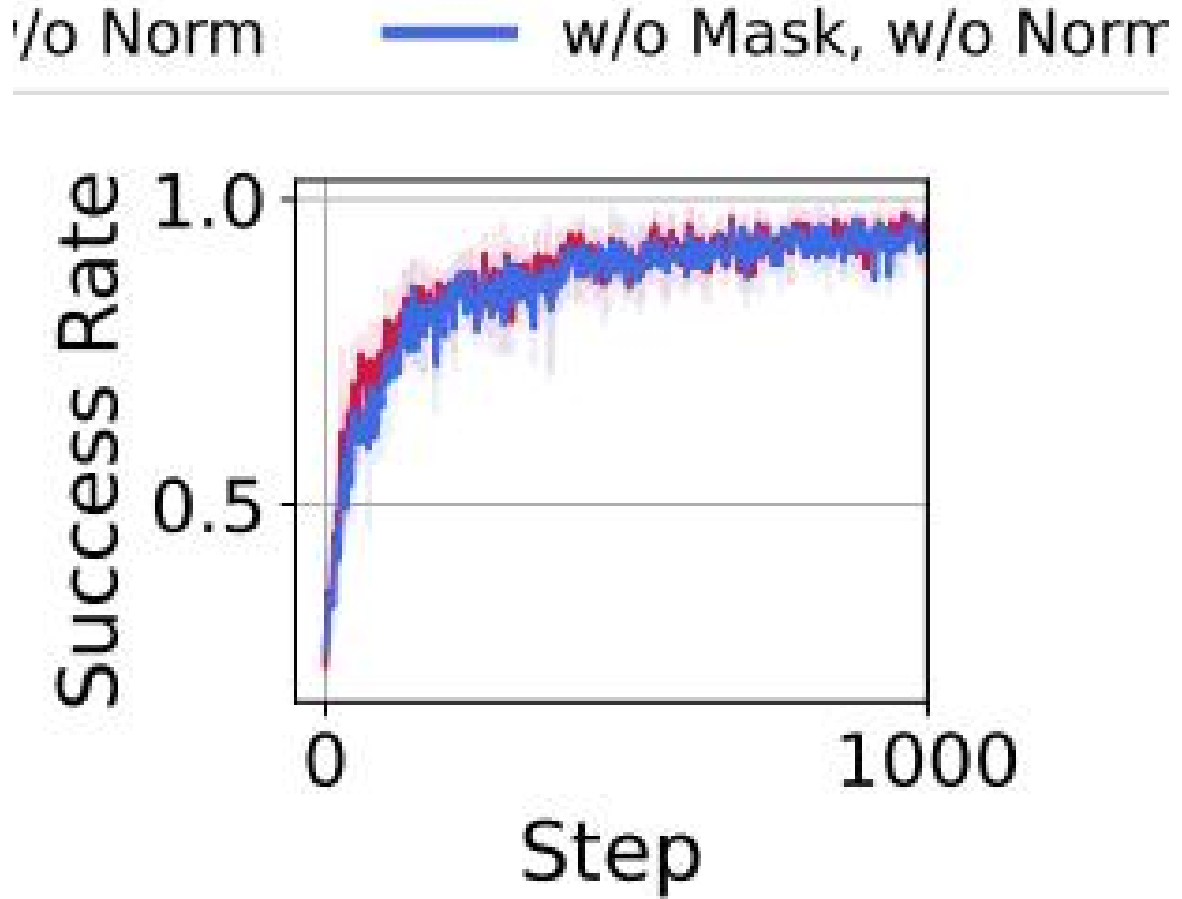


图 22: GRPO 中 valid action mask 的消融实验。(b) ManiSkill 上的成功率。图 14: 基于 OpenVLA-OFT 的 GRPO valid action mask 消融实验。

(c) **Success rate filtering** 在某些 GRPO 设置下可以提升训练稳定性。如第 IV-C3 节所述，我们为 GRPO 实现了 success rate filter：当一个 group 中所有 trajectory 的累计奖励都相同，就将该 group 丢弃。该机制能够提高 GRPO 训练的稳定性。例如，在 OpenVLA 的 ManiSkill 设置中，不使用该 filter (“w/o Filter”) 时，训练在大约 step 400 附近会出现明显崩塌；而启用 filter (“w/ Filter”) 后，这一问题得到明显缓解。不过，filter 的收益并不具有普遍性：在 OpenVLA-OFT 的 ManiSkill 设置以及 OpenVLA-OFT 的 LIBERO-Goal 设置中，它的作用就不那么显著。



图 23: 真实世界部署 (deployment)。

6.5 D. 真实世界实验

我们进行了一个初步的真实世界实验，用于评估：在仿真中通过 RL 获得的性能提升，能否迁移到物理部署中。

具体而言，我们考虑基于 Open-VLA 的关抽屉任务，并比较仅使用 SFT 训练的模型与在仿真中进一步通过 RL 优化后的模型。SFT 模型使用 20 条真实世界专家 demonstration 和 1000 条仿真轨迹进行训练。RL 模型从 SFT checkpoint 初始化，并在仿真环境中继续通过 RL 训练。为减小 sim-to-real gap，我们对真实环境中的相机位姿与机械臂配置进行了标定，使其尽可能与仿真环境保持一致。

在真实部署中，SFT 模型与 RL 模型的成功率 (success rate) 均为 8/10。然而，RL 模型能够以更高的效率完成任务 (图 15)。我们观察到，SFT 模型倾向于模仿次优的人类 teleoperation 行为，例如在推动抽屉之前先进行旋转，并且有时会预测接近于零的动作。相比之下，经 RL 训练的 policy 学到了更加直接且流畅的推动行为。尽管该实验规模仍然有限，这些结果表明，基于仿真的 RL 有望提升真实世界中 policy 执行的质量与效率。

7 VI. 结论

本文提出了 RLinf-VLA，一个面向基于强化学习的 Vision-Language-Action 模型训练的统一且高效的框架。RLinf-VLA 集成了多种 simulator、算法以及 VLA 架构，并提供灵活的执行模式与系统级优化，从而显著提升训练效率。大量实验表明，基于 RLinf-VLA 训练的模型在广泛的模拟任务上均取得了较高性能。

此外，我们的研究进一步总结了适用于 PPO 和 GRPO 的可操作实践，为未来基于强化学习的 VLA 训练研究提供参考。通过开源并持续维护 RLinf-VLA，我们为社区提供了一个能够加速、规范化并规模化推进具身智能研究的基础平台。

7.1 致谢

本工作得到了国家自然科学基金 (No.62406159、62325405)、中关村学院 (Grant No. C20250301)、北京信息科学与技术国家研究中心 (BNRist) 以及北京未来芯片创新中心的支持。

8 参考文献

参考文献条目较多，此处保留原论文英文文献列表，请参阅原文 PDF 的 References 一节。

9 RLinf-VLA 的附录

10 VII. 理论背景

10.1 A. RL 的定义

定义 1 (Return) . Return R_t 表示从时刻 t 开始的、按 γ 折扣的累积奖励：

$$R_t = \sum_{k=0}^{\infty} \gamma^k r(s_{t+k}, a_{t+k}). \quad (2)$$

定义 2 (Value Function) . Value Function $V_{\pi}(s)$ 表示：从状态 s 出发并遵循 policy π 时，期望获得的 return：

$$V_{\pi}(s) = \mathbb{E}_{\pi}[R_t \mid s_t = s], \quad (3)$$

其中, t 表示一个任意时刻, 此时 agent 处于状态 s_t 。

定义 3 (Action-Value Function) . Action-Value Function $Q_\pi(s, a)$ 表示: 在状态 s 下执行动作 a , 并在此后遵循 policy π 时, 期望获得的 return:

$$Q_\pi(s, a) = \mathbb{E}_\pi[R_t \mid s_t = s, a_t = a], \quad (4)$$

其中, t 表示一个任意时刻, 此时 agent 处于状态 s 并采取动作 a 。

定义 4 (Advantage Function) . Advantage Function $A_\pi(s, a)$ 用于刻画: 在状态 s 下采取动作 a , 相对于该状态期望 value 的优势程度:

$$A_\pi(s, a) = Q_\pi(s, a) - V_\pi(s). \quad (5)$$

10.2 B. PPO 简介

PPO [36] 是机器人学中应用最广泛的强化学习算法之一。PPO 通过将 policy 更新约束在 trust region 内来提升训练稳定性, 从而避免过大的参数变化导致学习过程失稳。这一机制通过 clipped surrogate objective 实现, 在保持 sample efficiency 的同时平衡 exploration 与 exploitation。由于兼具鲁棒性与实现简洁性, PPO 已成为机器人强化学习中的标准 baseline, 也是本文框架的基础。

在 PPO 中, advantage function 通常使用广义优势估计 (Generalized Advantage Estimation, GAE) [35] 来计算:

$$\hat{A}_t = \sum_{k=0}^{T-t-1} (\gamma\lambda)^k (r_{t+k} + \gamma V(s_{t+k+1}) - V(s_{t+k})), \quad (6)$$

其中, r_t 表示时刻 t 的奖励, $V(s)$ 为 value function, γ 是折扣因子, λ 用于控制偏差-方差权衡, T 表示一个 episode 的时间跨度。

PPO 的优化目标定义为:

$$J^{\text{PPO}}(\theta) = \mathbb{E}_t \left[\min(\rho_t(\theta) \hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right], \quad (7)$$

其中

$$\rho_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\theta_{\text{old}}}(a_t \mid o_t)}, \quad (8)$$

$\pi_{\theta_{\text{old}}}$ 表示 rollout policy, ϵ 为 clipping 参数。²

10.3 C. GRPO 简介

GRPO [38] 是近期提出的一种 policy optimization 变体, 目标是简化强化学习训练流程。与 PPO 不同, PPO 需要同时训练 policy model 和 value model, 而 GRPO 通过基于组内轨迹相对比较的方式, 去除了对显式 value function 的依赖。这样的设计降低了整体模型复杂度, 也避免了 value estimation 可能带来的误差。因此, GRPO 为 PPO 提供了一种轻量但有效的替代方案, 尤其适合大规模 VLA 训练这类同时关注效率与简洁性的场景。在 GRPO 中, importance ratio 可定义为:

$$\rho_t^{(i)}(\theta) = \frac{\pi_\theta(a_t^{(i)} \mid o_t^{(i)})}{\pi_{\theta_{\text{old}}}(a_t^{(i)} \mid o_t^{(i)})}, \quad (9)$$

其中, i 表示第 i 条轨迹。其 `advantage` 定义为:

$$\hat{A}_t^{(i)} = \hat{A}_t^{(i)} = \frac{\mathcal{R}^{(i)} - \text{mean}(\{\mathcal{R}^{(j)}\}_{j=1}^G)}{\text{std}(\{\mathcal{R}^{(j)}\}_{j=1}^G)}, \quad \forall t \in \{0, 1, \dots, |\tau^{(i)}|\}, \quad (10)$$

其中, $\mathcal{R}^{(i)}$ 表示轨迹 $\tau^{(i)}$ 的总奖励。需要注意, $\mathcal{R}$ 与折扣回报 R 并不相同, G 表示组大小。给定从样本缓冲区 D 中采样得到的初始观测 o_0 , 行为模型 $\pi_{\theta_{\text{old}}}$ 会生成 G 条轨迹 $\{\tau_i\}_{i=1}^G$ 。GRPO 的优化目标为:

$$J^{\text{GRPO}}(\theta) = \mathbb{E}_{o_0 \sim D, \{\tau^{(i)} \sim \pi_{\theta_{\text{dat}}}\}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|\tau^{(i)}|} \sum_{t=1}^{|\tau^{(i)}|} \min \left(\rho_t^{(i)}(\theta) \hat{A}_t^{(i)}, \text{clip} \left(\rho_t^{(i)}(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t^{(i)} \right) \right], \quad (11)$$

其中, $|\tau^{(i)}|$ 表示轨迹 $\tau^{(i)}$ 的长度, ϵ 为 `clip` 参数。

11 RLINF-VLA 的实现细节

11.1 A. 统一接口

统一接口包含两类功能: 核心函数与工具函数。

a) 核心函数 我们实现了标准的 Gym 风格 API, 包括 `reset` 和 `step`。此外, `step` 函数额外支持 `auto_reset` 选项: 启用后, 任何终止 (`terminate`) 或截断 (`truncate`) 的子环境都会被自动重置, 从而避免环境空转, 提高采样效率。

遵循 ManiSkill [43] 的约定, 我们还支持 `ignore_terminations` 选项。启用后, 系统会忽略 `termination` 信号, 只保留 `truncation` 信号; 也就是说, 只有当达到 `episode` 的最大长度时, 当前 `episode` 才会结束。借助这一特性, 我们可以灵活支持不同的实现变体, 例如 *Partial Reset* 和 *Valid Action Mask*。

除此之外, 我们进一步扩展了接口, 提供 `chunk_step` 函数以处理 `action chunk`。与其通过重复调用 `step` 对整个 `chunk` 进行朴素循环, 该函数会更加谨慎地管理 `episode` 的终止边界。具体支持两种模式:

1. 当某个子环境在 `chunk` 内部完成时, 立即执行 `reset`;
2. 推迟 `reset`, 直到整个 `chunk` 执行完毕后再统一处理。

这种灵活性能够在使用 `chunked actions` 时, 正确处理 `episode` 的边界。

b) 工具函数 工具函数为训练与评估提供了便捷支持。例如, 可视化工具能够在 `rollout` 或评估过程中方便地生成视频。此外, 我们还提供了一些特定算法所需的专用工具函数。以 GRPO 为例, 它要求同一组内的所有环境共享相同的初始状态; 这一点可以通过设置 `use_fixed_reset_state_ids=True` 来保证。

11.2 B. GPU Allocation Modes

a) GPU 分配模式的配置：我们的框架提供了一个简洁的配置接口，用户无需显式指定具体的分配模式，即可灵活选择 GPU 分配策略。具体而言，用户只需通过以下字段为各个组件分配 GPU ID：

- 用于 Simulator 的 `cluster.component_placement.env`,
- 用于 Generation 的 `cluster.component_placement.rollout`,
- 用于 Train 的 `cluster.component_placement.actor`。

此外，还可以通过以下配置项，分别为每个组件独立启用或关闭 offloading：

- `env.enable_offload`,
- `rollout.enable_offload`,
- `actor.enable_offload`。

最后，细粒度的流水线并行由参数 `rollout.pipeline_stage_num` 控制。当该值大于 1 时，会将同一块 GPU 上的每个 simulator 实例划分为多个 pipeline stage；当该值为 1 时，则禁用流水线并行。

11.3 C. 设计选择细节

1) 多粒度支持的细节：正文中提到三种粒度层级：token-level、action-level 和 chunk-level。图 16 给出了一个示意例子。一个 action chunk 由多个原子动作（atomic action）组成，而原子动作是在每个时间步由模拟器实际执行的控制信号。进一步地，每个原子动作又由多个 token 构成，其中每个 token 对应一个动作维度。

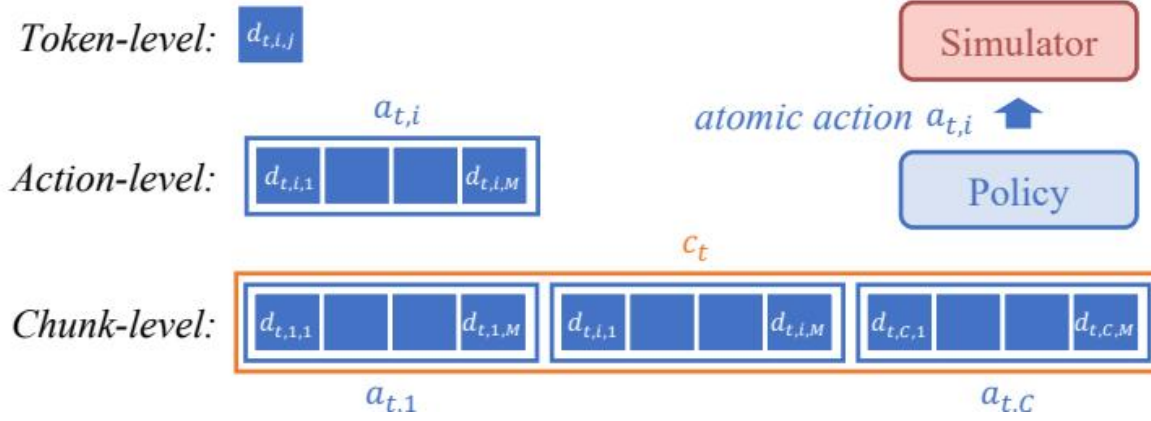


图 24: 不同 log-probability 粒度的示意图。

advantage 计算粒度与 log-probability 粒度之间所有支持的组合如表 II 所示。

表 2: advantage 与 log-probability 粒度的支持组合。

	Chunk-level	Action-level	Token-level
Advantage Chunk-level	✓	✓	✓
Action-level	×	S	✓

2) 按轨迹长度归一化损失的细节：正如正文所述，在 Valid Action Mask 设置下，我们按轨迹长度对 policy loss 进行归一化。具体来说，若一条轨迹 τ_i 有 T_i^{succ} 个有效时间步，则目标函数中每个时间步的贡献都会按 $1/T_i^{\text{succ}}$ 进行缩放。在该设置下，GRPO 目标函数写为：

$$J^{\text{GRPO}}(\theta) = \mathbb{E}_{o_0 \sim D, \{\tau_i\} \sim \pi_{\text{odd}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{T_i^{\text{succ}}} \sum_{t=1}^{T_i^{\text{succ}}} \min \left(\rho_{i,t}(\theta) \hat{A}_i, \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right]. \quad (12)$$

这样可以避免较长轨迹在梯度中占据不成比例的主导地位，并保证不同长度轨迹之间的学习更加平衡。

12 IX. EXPERIMENT SETUP

12.1 A. 性能实验细节

本节给出高性能实验的训练与评测细节，包括评估指标、基础模型选择以及训练超参数设置。除非另有说明，所有曲线均使用高斯滤波器($\sigma = 1$)进行平滑；成功率(success rate)按照“success_once”准则计算，即只要某个 episode 中至少一次达到成功状态，就记为成功。

1. **训练所用基础模型：**对于 ManiSkill 中的 OpenVLA (Base)，我们采用 RL4VLA [25] 提供的预训练 checkpoint OpenVLA (Base)。对于 ManiSkill 中的 OpenVLA-OFT (Base)，我们基于从

“PutOnPlateInScene25Main-v3” 任务采集的 motion planning 数据, 自行通过 LoRA 微调得到 OpenVLA-OFT (Base)。

对于 LIBERO 的 130 个任务中的 OpenVLA-OFT (Base), 我们使用 LIBERO 中全部五个任务套件的 demonstrations, 通过 LoRA 对 OpenVLA-OFT 进行微调。

对于 RoboTwin 中的 OpenVLA-OFT (Base), 我们采用 SimpleVLA-RL [23] 提供的预训练 checkpoint OpenVLA-OFT (Base)。此外, 我们还复用了 SimpleVLA-RL [23] 的训练与评测随机种子 (train/eval seeds)。

2. **训练超参数设置:** 表 III 列出了不同实验所使用的具体超参数。

3. **评估指标:** 对于监督微调得到的模型, 我们在评估时设置 `do_sample=False`。对于经 RL 训练的模型, 我们在评估时设置 `do_sample=True`, 并对每个模型进行三次评估。最终报告成功率的均值和标准差。(目前仅 LIBERO 设置报告了标准差。)

12.2 B. Efficiency 实验细节

1) **Efficiency 实验的超参数:** 不同 offload 配置的启用方式见表 4, GPU 分配方式的配置见表 4。训练所用超参数见表 3。

表 3: 训练超参数设置。

Parameter	OpenVLA [20] ManiSkill [43]		OpenVLA-OFT [21] ManiSkill		OpenVLA-OFT LIBERO [24]		OpenVLA-OFT RoboTwin [9]
	PPO	GRPO	PPO	GRPO	GRPO	GRPO	
# Parallel Envs	128	256	128	256	64		128
Max Episode Steps	80	80	80	80	512		200 ^a
Max Steps per Rollout Epoch	160	80	160	80		512	200 ^a
Group Size	1	8	1	8		8	8
Rollout Epoch	1	1	1	1		64	8
Partial Reset	True	False	True	False		False	False
Valid Action Mask	False	True	False	True		True	True
Action Chunk Size	1	1	8	8	8		25
Global Batch Size	640	640	640	640		16384	1024
Micro Batch Size	40	40	40	40		32	32
is_lora [16]	True	True	True	True			True
Learning Rate	1e-4	1e-5	1e-4	1e-4		2e-5	1e-4
adam_eps	1e-8	1e-8	1e-8	1e-8		1e-8	1e-5
Clip Ratio (ϵ)	(0.2, 0.28)	(0.2, 0.28)	(0.2, 0.28)	(0.2, 0.28)		(0.2, 0.28)	(0.2, 0.28)
γ	0.99		0.99				
GAE λ	0.95		0.95				
Temperature (train)	1.0	1.0	1.0	1.0		1.6	1.6

^a 需要注意的是, RoboTwin 中的 “pick dual bottles” 和 “handover block” 两个任务分别需要 100 步和 400 步。

表 4: 不同 GPU 分配模式下 enable_offload 的设置。

	Simulator	Generation	Training
Disaggregated	False	False	False
Colocated	True	True	True
Hybrid	True	True	True

2) Efficiency 指标: 我们使用吞吐量 (throughput) 作为 Efficiency 的评估指标。具体而言, throughput 定义为一个训练 epoch 内 rollout 环境帧总数除以该 epoch 的总 wall-clock 时间; 后者可近似看作 rollout 时间与训练时间之和。

3) Efficiency 实验任务: 我们在 ManiSkill、LIBERO 和 RoboTwin 的全部任务上评估所提出的框架。

对于 ManiSkill, 我们选用 “PutCarrotOnPlateInScene-v2” 任务作为快速测试。该任务由 ManiSkill 中的 “PutCarrotOnPlateInScene-v1” 修改而来, 主要改动在 reset 函数上。在原始 reset 实现中, 为了确保所有物体保持静止, 会额外执行若干仿真步。然而, 这会使 reset 时间相比 step 函数显著增加, 并且与 partial reset 不兼容。我们的实际观察表明, 在 “PutCarrotOnPlateInScene” 任务中, 这些额外仿真步对物体状态的影响可以忽略不计, 因为胡萝卜的初始位置本身已经经过仔细校准。因此, 我们删除了 reset 函数中的额外仿真步。评测在 8、16 和 32 张 NVIDIA H100 (80GB) GPU 上进行, 并使用 256 个并行环境, 每个环境运行 80 步。

12.3 B. 效率实验细节（第 2 部分）

表 5: GPU 分配模式的超参数。

# GPU	分配模式 (Allocation Mode)	Simulator	Generation	Training
8 GPUs	Disaggregated	0–1	2–3	4–7
	Colocated	0–7	0–7	0–7
	Hybrid	0–3	4–7	0–7
	Disaggregated	0–3	4–7	8–15
16 GPUs	Colocated	0–15	0–15	0–15
	Hybrid	0–7	8–15	0–15
		0–7		
32 GPUs	Disaggregated Colocated	0–31	8–15 / 0–31	16–31 / 0–31
	Hybrid	0–15	16–31	0–31

对于 LIBERO, 我们采用 LIBERO-Long 任务集进行快速测试, 因为它是 LIBERO 中任务时长最长的任务集。对于 8、16 和 32 GPU 配置, 并行环境数量分别设为 64、128 和 256; 对应的 environment steps 分别设为 4096、2048 和 1024.³

对于 RoboTwin, 我们选择 `place_empty_cup` 任务进行快速测试。该任务要求机器人使用机械臂将空杯放置到杯垫上。对于 8、16 和 32 GPU 配置, 并行环境数量同样分别设为 64、128 和 256; 对应的 environment steps 分别设为 800、400 和 200。由于 rollout 的上限受 GPU 资源约束, 我们在 RoboTwin 中采用与 LIBERO-Long 相同的 GPU 分配设置。

12.3.1 4) 效率实验的 baseline

对于 ManiSkill, 现有框架尚不支持多 GPU rollout 与训练。因此, 我们将朴素的 Disaggregated 分配模式作为 baseline, 并将其与本文提出的 Colocated 模式和 Hybrid 模式进行比较; 其中, Hybrid 模式同时包含 one-stage 和 two-stage 的细粒度流水线配置。

对于 LIBERO 和 RoboTwin, 我们与 SimpleVLA-RL [?] 进行比较。SimpleVLA-RL 是一个基于 VeRL 构建、面向 VLA 模型强化学习训练的开源框架。由于 SimpleVLA-RL 仅支持 Colocated 模式, 因此我们将其作为 baseline, 并进一步评估我们自己的 Colocated 模式和 Hybrid 模式（包含 one-stage 与 two-stage 的细粒度流水线配置）。

我们未报告 LIBERO 和 RoboTwin 上的 Disaggregated 模式结果, 因为在训练过程中, Training 组件不会与 rollout 过程复用同一组 GPU。因此, 即便将 offload-onload 开销计入在内, Hybrid

(one-stage) 配置的性能仍优于 Disaggregated 模式。

13 X. 补充实验结果

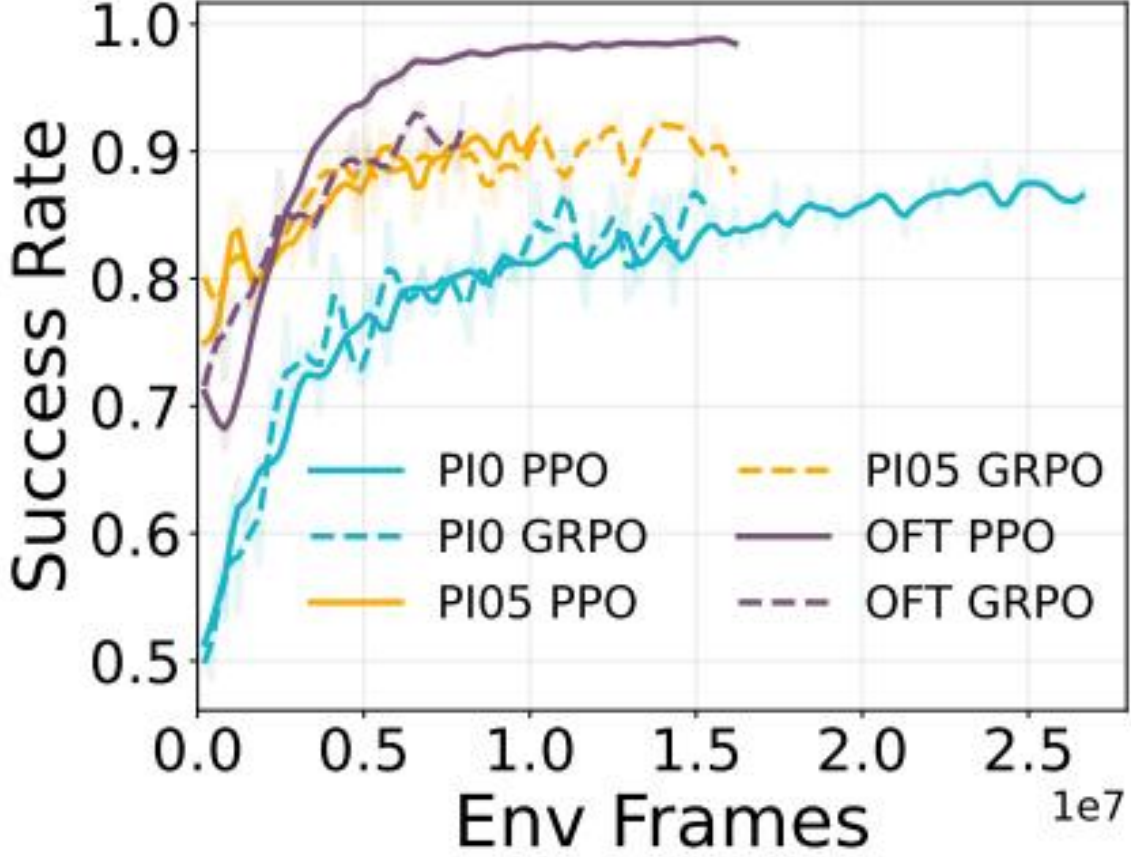


图 25: RLinf-VLA 在不同模型与算法上的可扩展性。(a) RoboTwin (Place Empty Cup) ; (b) ManiSkill (PickCube), MLP; (c) LIBERO, DSRL。

13.1 A. RLinf-VLA 的可扩展性

1) **模型可扩展性:** RLinf-VLA 还支持: (a) flow-matching 模型; (b) 小模型。图 17(a) 和图 17(b) 分别给出了在 π -series 模型与 MLP 上的结果。对于 π -series 模型, 我们采用了 [8] 中的 GRPO 和 PPO。

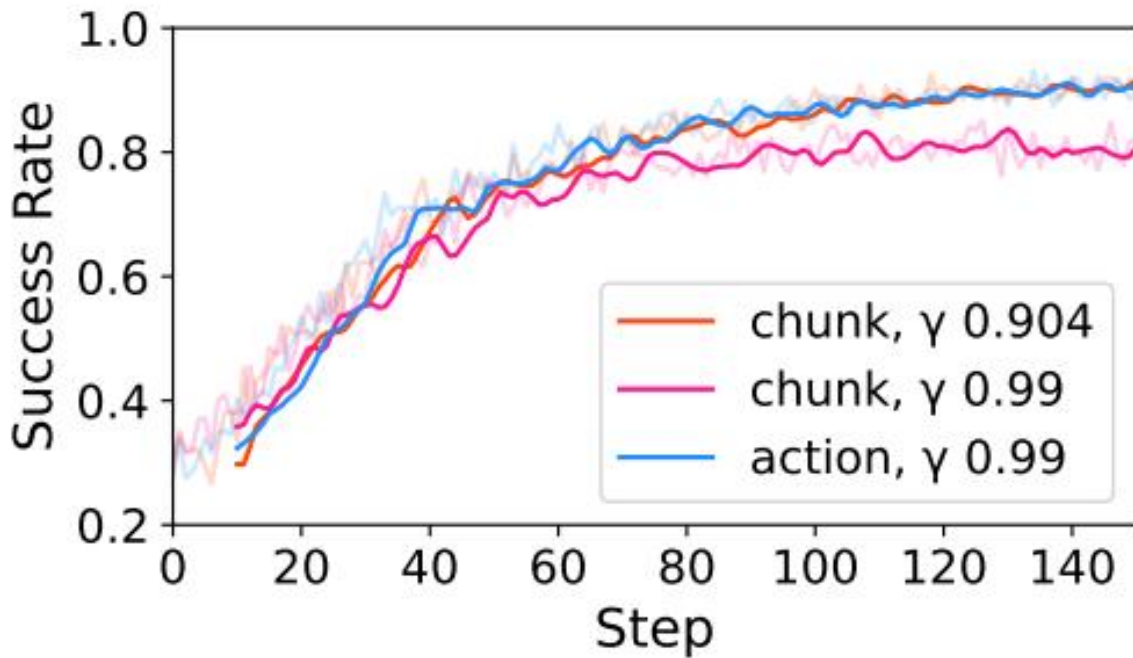
2) **算法可扩展性:** 我们的框架还在多种设定下支持 SAC 等 off-policy 方法。然而, 将 SAC 应用于 VLA 模型仍然是一个开放问题, 尤其是在实现稳定训练方面。尽管如此, 整个 pipeline 已经可以完整运行。我们通过在 π_0 上使用 DSRL [45] (图 17(c)) 以及在 MLP 上使用 SAC (图 17(b)) 展示了这一能力。

13.2 B. ManiSkill 上的 OOD 评估

表 VI 给出了 ManiSkill 实验的详细结果。

表 6: ManiSkill 上 Out-Of-Distribution 评估结果的详细分解。表中数值表示成功率 (success rate, %)。

Method	Out-Of-Distribution		
	Vision	Semantic	Execution
OpenVLA (Base)	38.75	35.94	42.11
OpenVLA (RLinf-GRPO)	74.69	72.99	77.86
OpenVLA (RLinf-PPO)	82.03	78.35	85.42
OpenVLA-OFT (Base)	27.73	12.95	11.72
OpenVLA-OFT (RLinf-GRPO)	84.69	45.54	44.66
OpenVLA-OFT (RLinf-PPO)	92.11	64.84	73.57

图 26: 在 ManiSkill 上, 基于 OpenVLA-OFT 的 action-level 与 chunk-level PPO 训练中, 折扣因子 γ 的消融实验结果: (a) 成功率 (success rate)。

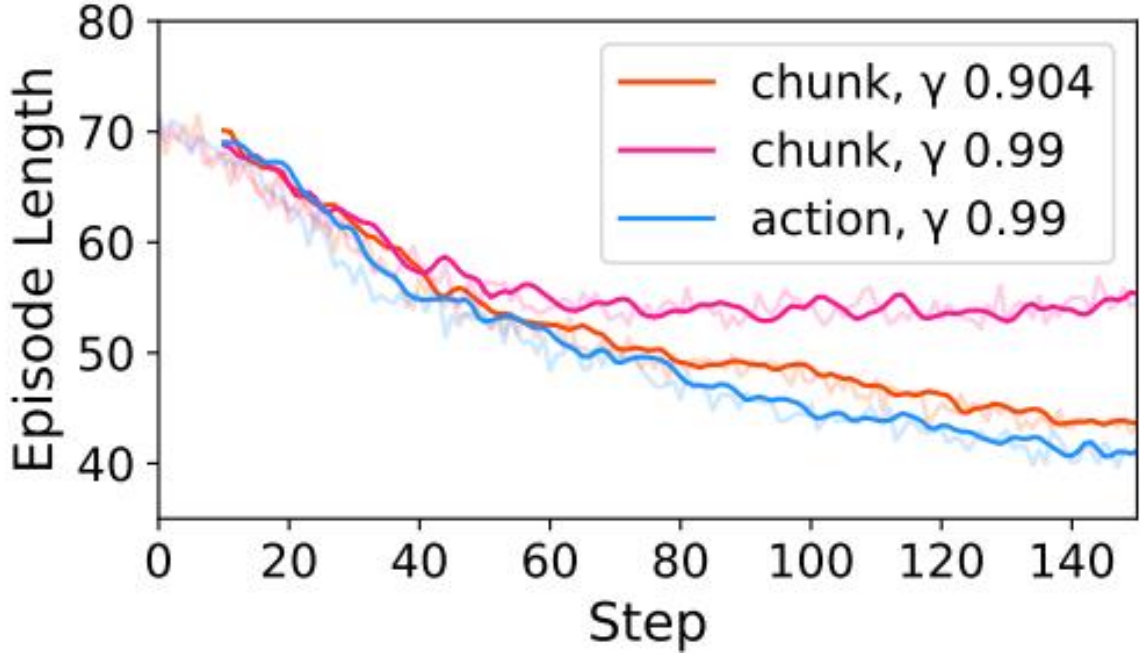


图 27: 在 ManiSkill 上, 基于 OpenVLA-OFT 的 action-level 与 chunk-level PPO 训练中, 折扣因子 γ 的消融实验结果: (b) 轨迹长度 (episode length)。

13.3 C. 消融实验

a) 带 action chunk 的: 在 chunk 级别上, 我们将整个 action chunk 视为一个单独的 RL action。这样的建模方式要求谨慎选择折扣因子 γ 。实验表明, 只有在折扣因子经过正确对齐时, chunk-level 训练才能取得与 action-level 训练相当的性能。具体而言, 若 chunk 大小为 c , 则折扣因子应满足

$$\gamma_{\text{chunk}} = \gamma_{\text{action}}^{1/c}.$$

实验结果如图 18 所示。在我们的设定中, chunk 大小为 $c = 8$ 。因此, 当采用 $\gamma_{\text{chunk}} = 0.904 \approx 0.99^{1/8}$ 进行 chunk-level 训练时, 其性能与采用 $\gamma_{\text{action}} = 0.99$ 的 action-level 训练相当。相反, 若直接使用 $\gamma_{\text{chunk}} = 0.99$, 性能会明显变差。这说明在 chunk-level 建模下, 对折扣因子进行恰当重缩放十分重要。

有趣的是, 这一现象也揭示了具身 RL 与面向 LLM 的 RL 之间的一个重要差异。在 LLM 场景中, 折扣因子通常设为 $\gamma = 1$, 因为序列生成天然偏向长时程的 credit assignment。相比之下, 具身 RL 往往更受益于较小的折扣因子。直观来看, 较小的 γ 会鼓励更短时程的行为以及更快的任务完成, 这正是机器人操纵任务所期望的性质。如图 18b 所示, 较小的 γ 往往对应更短的平均轨迹长度。此外, 当启用带 partial reset 的 PPO 时, 更短的轨迹还能进一步提高数据采集效率, 因为环境可以更频繁地 reset 并恢复执行。

b) 成功率过滤 (success rate filtering): 如图 19 所示, 我们在 GRPO 中实现了一种成功率过滤机制: 当某一组内所有轨迹的累积奖励完全相同时, 就丢弃该组数据。该机制能够提升 GRPO 训练的稳定性。例如, 在 OpenVLA ManiSkill 设定下, 不使用过滤器的训练配置 (⁶⁶w/o Filter³) 会在约 400 step 附近出现明显崩塌, 而启用过滤器 (“w/ Filter”) 后, 这一问题得到了较大缓解。不过, 这种收益并非普遍存在: 在 OpenVLA-OFT ManiSkill 设定以及 OpenVLA-OFT LIBERO-Goal 设定中, 过滤器的效果就没有那么显著。

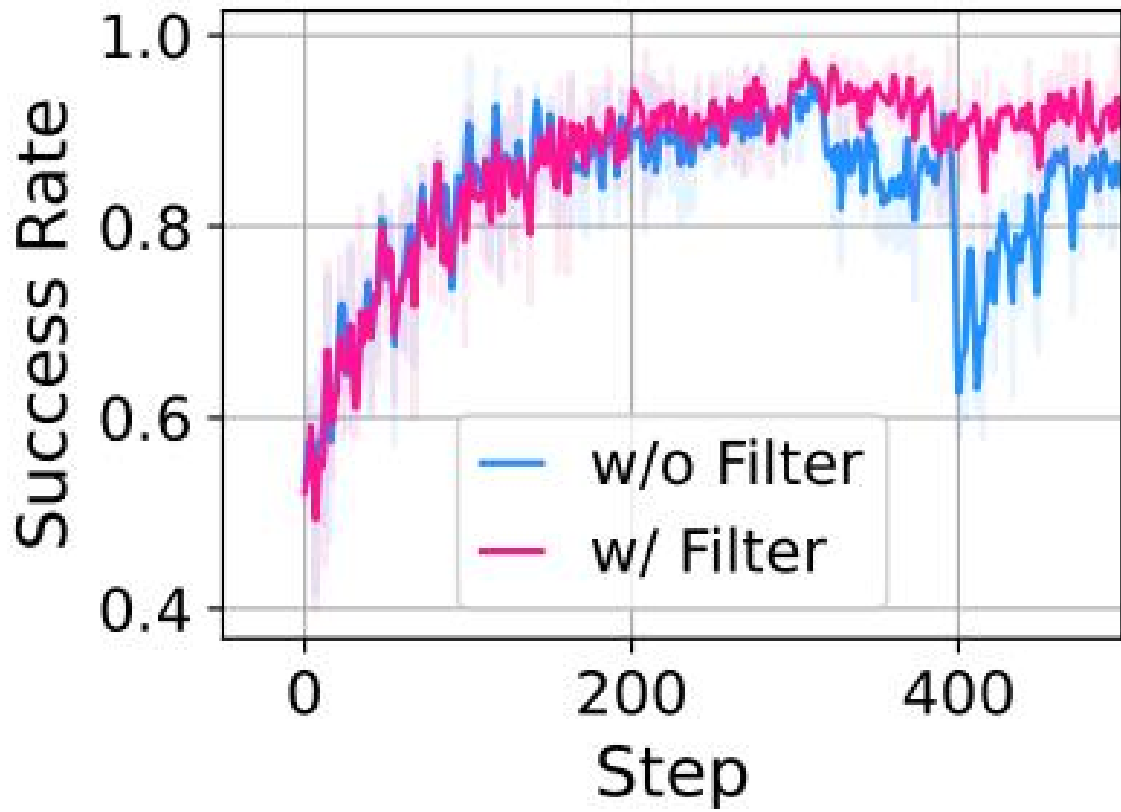


图 28: (a) OpenVLA, ManiSkill

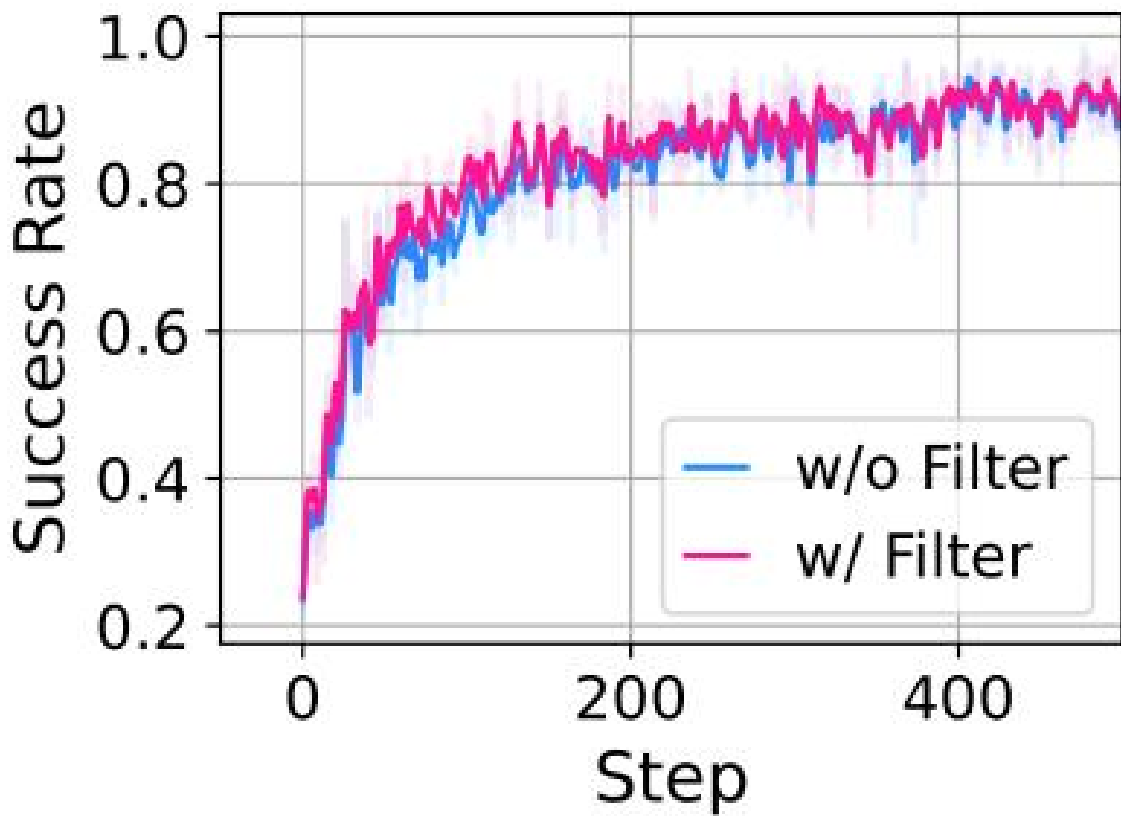


图 29: (b) OpenVLA-OFT, ManiSkill

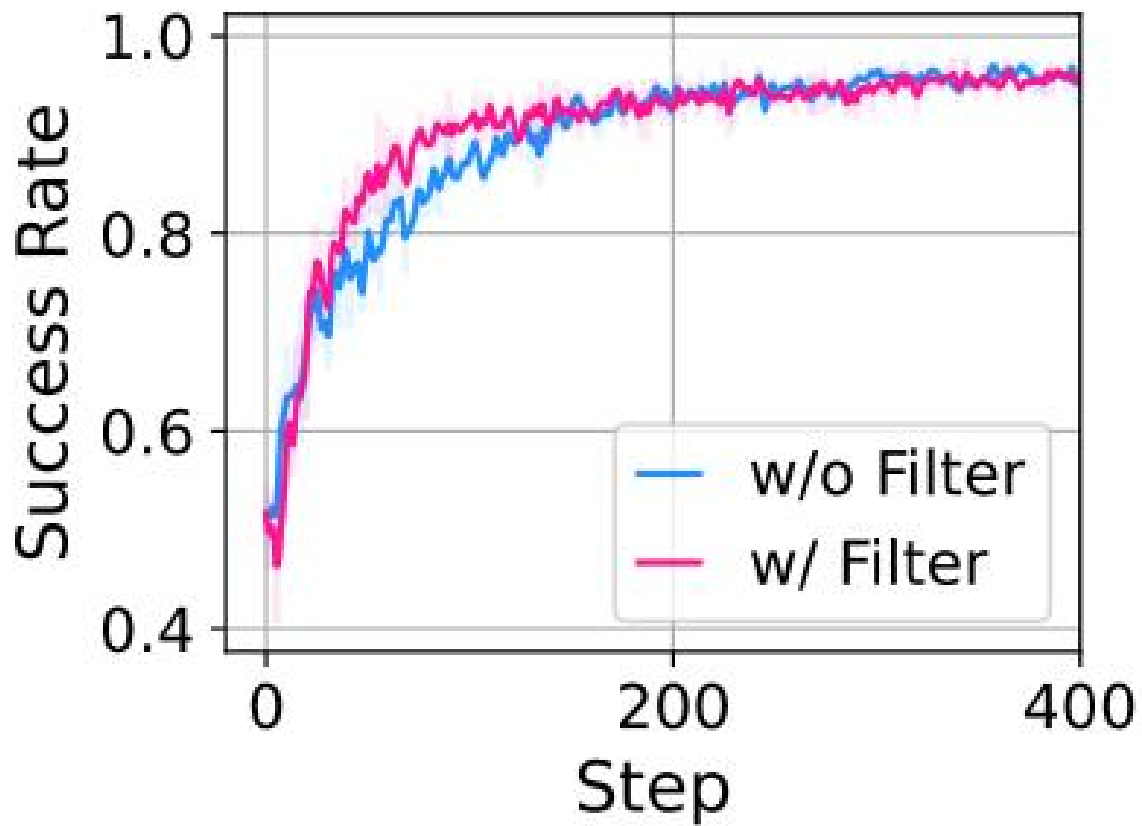


图 30: (c) OpenVLA-OFT, LIBERO-Goal。图 19: 不同设定下成功率过滤的消融实验。

c) rollout 数据规模的影响： rollout batch size 对 RL 性能有显著影响，尤其是对于 PPO 和 GRPO 这类 on-policy 算法。总体而言，每个 epoch 使用更大的 rollout 数据，能够使每轮训练迭代中的 policy 改进更加充分。如图 20 所示，当以训练 epoch 为横轴进行评估时，更大的 rollout 规模始终对应更高的成功率（success rate）。

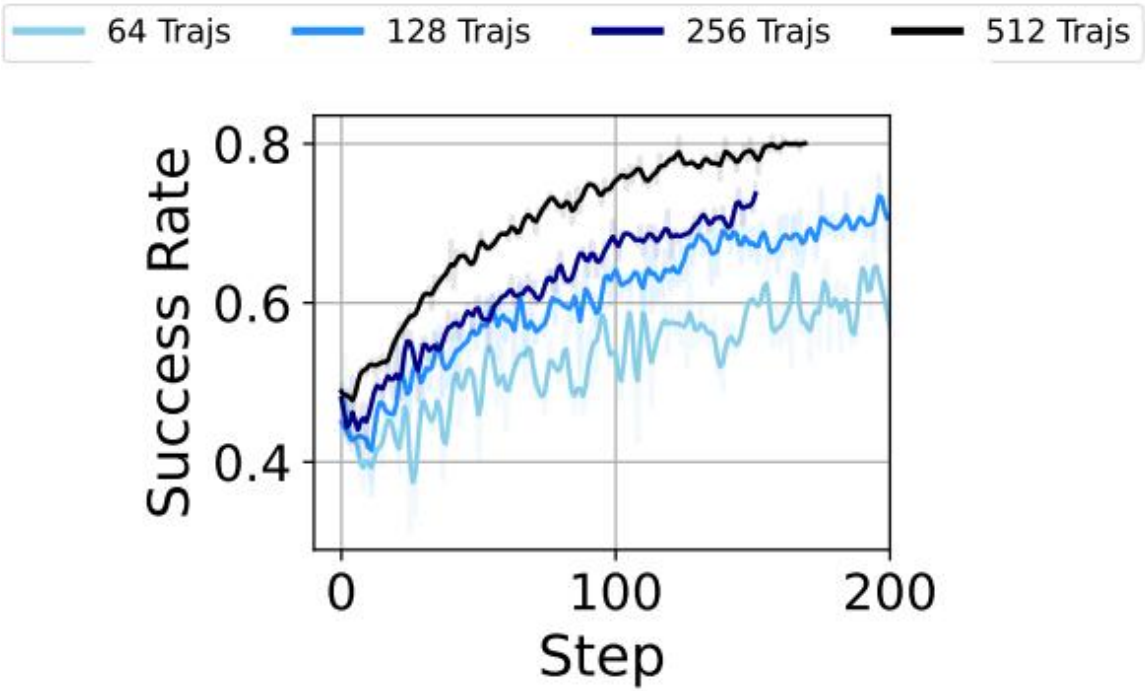


图 31: (a) PPO, LIBERO-Long

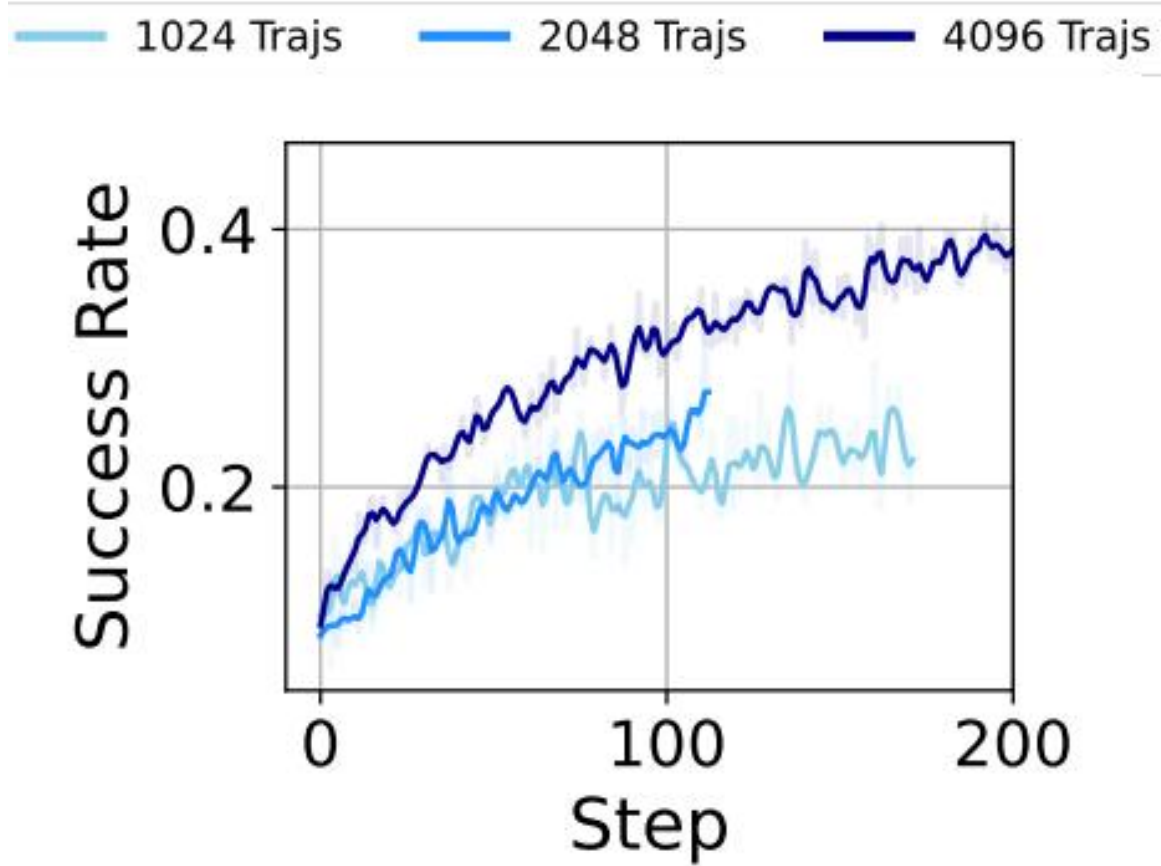


图 32: (b) GRPO, LIBERO 130 个任务。图 20: rollout 数据规模的消融实验。颜色越深, 表示 rollout 数据集越大。

d) 使用 LoRA 未必直接影响性能,但通常需要不同的超参数: 图 21 给出了使用与不使用 LoRA [16] 时的训练曲线。横轴为训练 epoch 数, 纵轴为 ManiSkill 上的成功率。整体来看, 两组曲线较为相似, 说明 LoRA 本身并不会显著改变性能。然而, 是否使用 LoRA 会影响最优超参数的选择。以 GRPO 实验 (图 21b) 为例, 当学习率固定为 1×10^{-4} 时, 使用 LoRA 可以正常提升性能, 而不使用 LoRA 时, 成功率会崩塌到零。相对地, 当学习率降低为 1×10^{-5} 时, 不使用 LoRA 的设定也能够实现稳定提升。这些结果表明, 不同的 LoRA 配置可能需要分别进行超参数调优。

13.4 C. 消融实验 (第 2 部分)

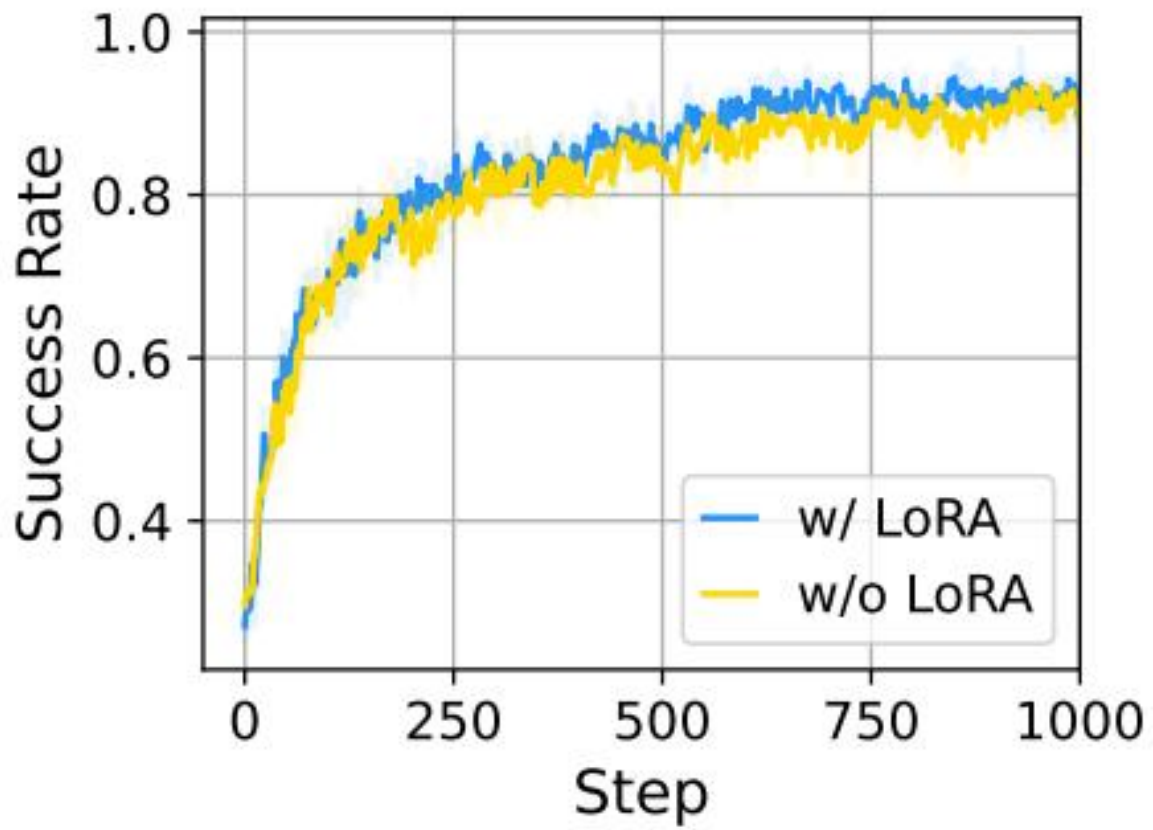


图 33: (a) PPO

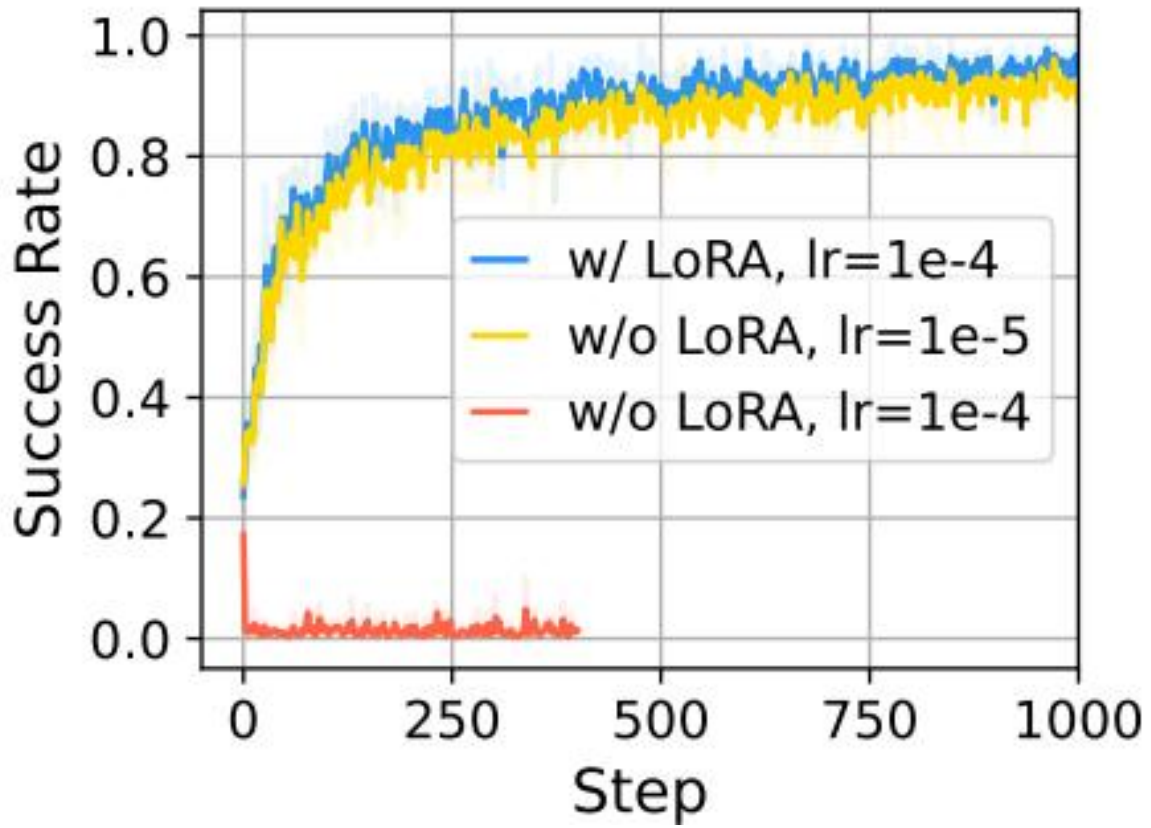


图 34: (b) GRPO

图 35: 图 21: 在 ManiSkill 中基于 OpenVLA-OFT 对 LoRA 的消融实验。